L01, Group 06
Melody Yang 400307007
Cynthia Sun 400238765

McMaster University

MECHTRON 3TB4

Lab Tutorial 5 Report

Lab 5 Prelab Report

Date: April 3, 2026

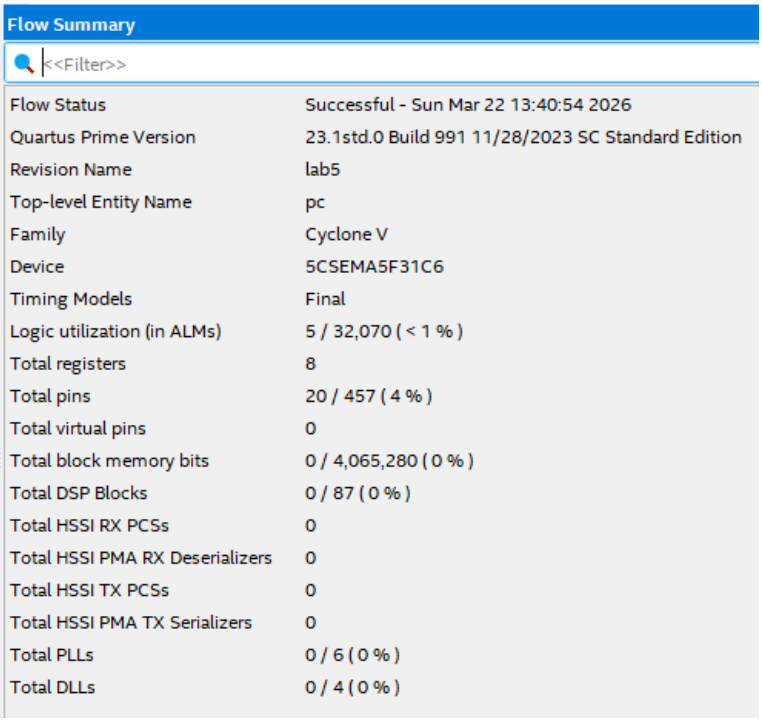
Lab Tutorial 5 Report

Overview

The goal of this lab tutorial is to integrate and validate the modules required for the datapath system. The modules were tested individually before they were combined into a single functional design, ensuring that the connections were correct based on the system block diagram provided. Simulations were performed using the waveform generator for each module and for the specific instructions, including ADDI, SUBI, and MOVR. The behaviour of the registers R0, R2 (position), and R3 (delay) were observed to confirm that the instructions were correctly executed. Waveform simulations were used for debugging by tracing signal values and verifying the interactions between different modules.

Program Counter (PC)

The Program Counter is an 8-bit register that serves as a pointer for the processor by storing the memory address of the next instruction to be fetched. The program counter is incremented by one during each clock cycle, allowing it to step through sequential instructions that are stored in memory. It can also jump to specific addresses when a branch instruction is triggered, allowing for the creation of loops and conditional logic for the motor control.



Flow Summary

<<Filter>>

Flow Status	Successful - Sun Mar 22 13:40:54 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab5
Top-level Entity Name	pc
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	5 / 32,070 (< 1 %)
Total registers	8
Total pins	20 / 457 (4 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 1. Screenshot of the Program Counter (PC) module compilation report.

Table 1. Summary for Components for the Program Counter (PC) Module in PreLab 5.

Feature	Quantity
Total number of logic elements	5
Total number of registers	8
Total number of pins	20
Max number of logic elements that can fit on FPGA	32,070

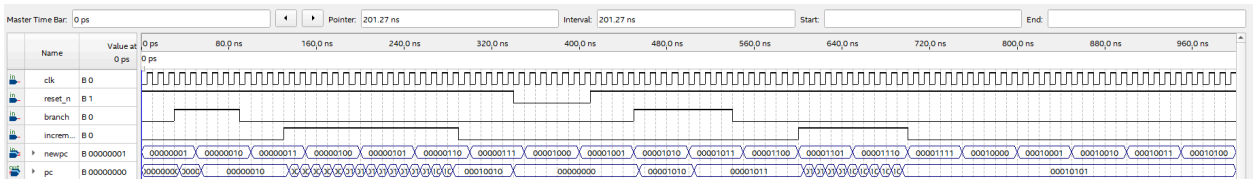


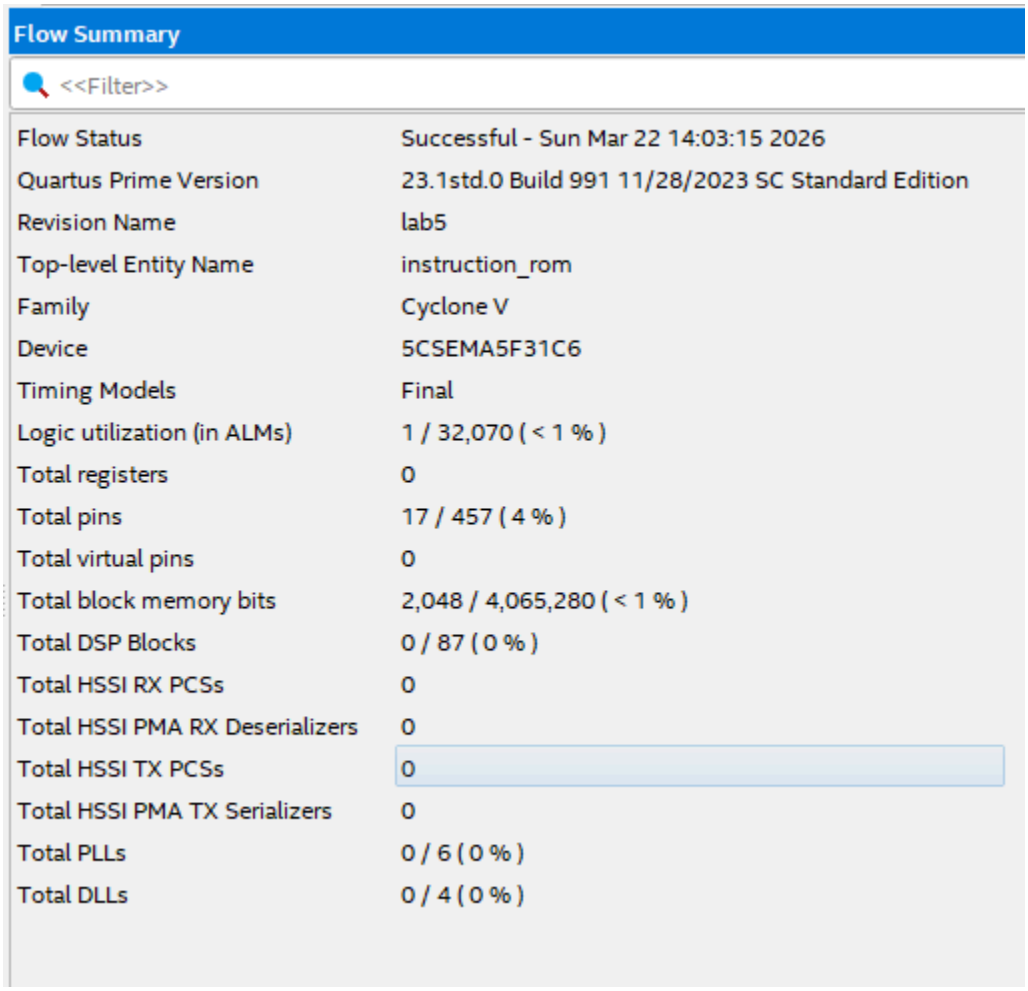
Figure 2. Screenshot of the waveform simulation for the Program Counter (PC) Module.

```
1  module pc (input clk, reset_n, branch, increment, input [7:0] newpc,
2            output reg [7:0] pc);
3
4     parameter RESET_LOCATION = 8'h00;
5
6     always@(posedge clk or negedge reset_n) begin
7
8        if(!reset_n)
9           pc <=RESET_LOCATION;
10
11       else if(branch)
12          pc <= newpc;
13
14       else if(increment)
15          pc <= pc + 8'b00000001;
16
17    end
18
19 endmodule
20
```

Figure 3. Screenshot of the Program Counter (PC) Module.

Instruction Memory

The instruction memory contains a sequence of machine instructions to be executed by the programmer. The machine instructions were created using the compiler.py by turning the assembly language from their respective .txt files. This results in an 8-bit binary instructions that can be used to program it. The max index of the instruction_rom.mif module was set to 256x8 synchronous memory implemented on-chip, indicating a total of 256 sets of 8-bit instructions are able to be programmed. However, this lab only utilized approximately 10-20 instructions for its different instruction sets based on the given prompts. The instruction_rom.v module was automatically generated using Quartus Prime. The binary instruction values correspond to the values shown in the decoder.v module. The instruction_rom.mif changes depending on the desired instruction set which is further discussed in the Prelab section.



Flow Summary

<<Filter>>

Flow Status	Successful - Sun Mar 22 14:03:15 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab5
Top-level Entity Name	instruction_rom
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	1 / 32,070 (< 1 %)
Total registers	0
Total pins	17 / 457 (4 %)
Total virtual pins	0
Total block memory bits	2,048 / 4,065,280 (< 1 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 4. Screenshot of the Instruction Memory module compilation report.

Table 2. Summary for Components for the Instruction Memory Module in PreLab 5.

Feature	Quantity
Total number of logic elements	1
Total number of registers	0
Total number of pins	17
Max number of logic elements that can fit on FPGA	32,070

Addr	+0	+1	+2	+3	+4	+5	+6	+7
0	01100000	01000010	01010011	01111100	01100000	01000011	01110100	11000001
8	11111111	01100000	01001100	01011111	01110100	11000001	11111111	10010101

Figure 5. Screenshot of the instruction_rom.mif instructions for the Prelab Spin3.mif

```
37  // synopsys translate_off
38  `timescale 1 ps / 1 ps
39  // synopsys translate_on
40  module instruction_rom (
41     address,
42     clock,
43     q);
44
45     input [7:0]  address;
46     input   clock;
47     output   [7:0]  q;
48  `ifndef ALTERA_RESERVED_QIS
49  // synopsys translate_off
50  `endif
51     tri1    clock;
52  `ifndef ALTERA_RESERVED_QIS
53  // synopsys translate_on
54  `endif
55
56     wire [7:0] sub_wire0;
57     wire [7:0] q = sub_wire0[7:0];
58
59     altsyncram  altsyncram_component (
60               .address_a (address),
61               .clock0 (clock),
62               .q_a (sub_wire0),
63               .aclr0 (1'b0),
64               .aclr1 (1'b0),
65               .address_b (1'b1),
66               .addressstall_a (1'b0),
67               .addressstall_b (1'b0),
68               .byteena_a (1'b1),
69               .byteena_b (1'b1),
70               .clock1 (1'b1),
```

Figure 6. Screenshot of the Instruction Memory Module.

Decoder

The Decoder module consists of combinational logic, which takes the top six bits of the 8-bit machine code from the instruction memory and decodes it to determine which control signals are active.The instructions correspond to the binary values shown in the instruction encoding figure below. Although the lab manual required the bits to be instruction[7:2], for the waveform simulator to work properly, it was set to instruction [7:0]. The decoder looks at the opcode and specific bit fields to turn on the correct signals across the system, depending on the instruction the machine code corresponds to. It controls when the Register File writes, the operation the ALU performs, and sets the select lines for all the multiplexers. The decoder essentially bridges the gap between the software commands and the hardware execution. As shown in the waveform for the decoder module, when one instruction is high such as BR, all the other instruction are set to low.

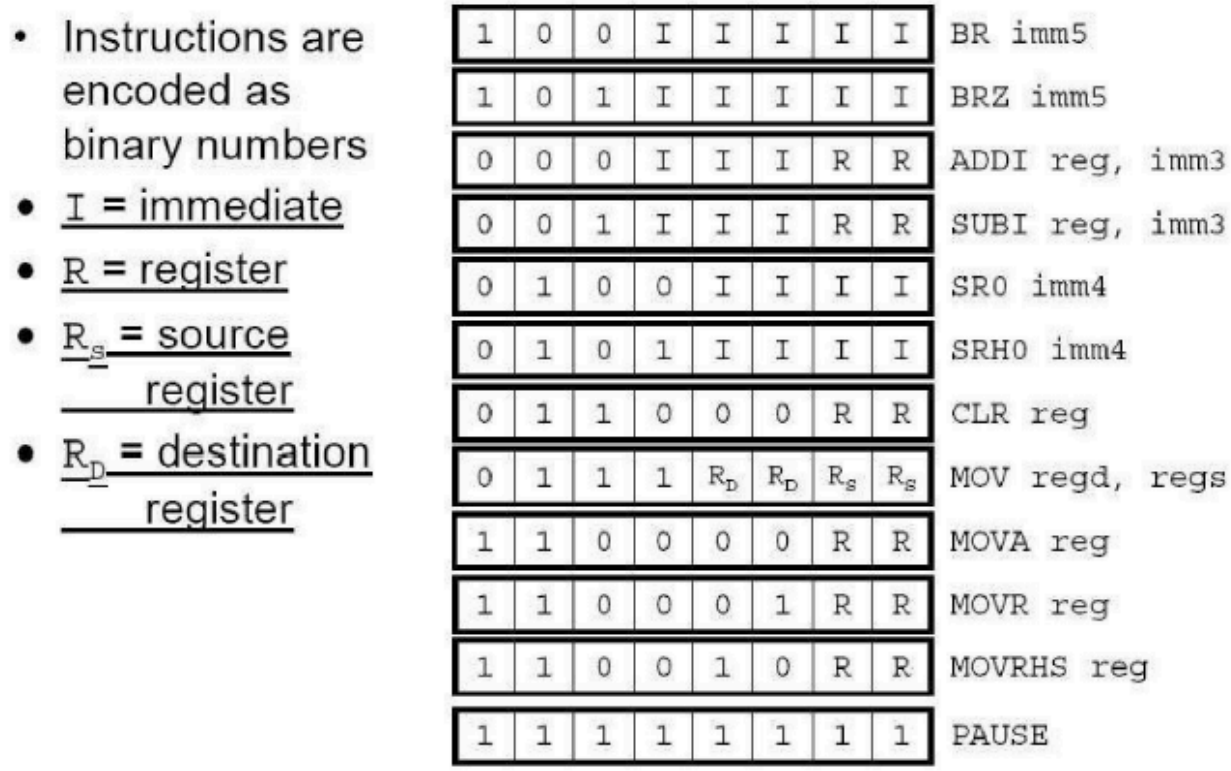


Figure 7. Instruction Encoding for the ASIP used in the decoder.v module

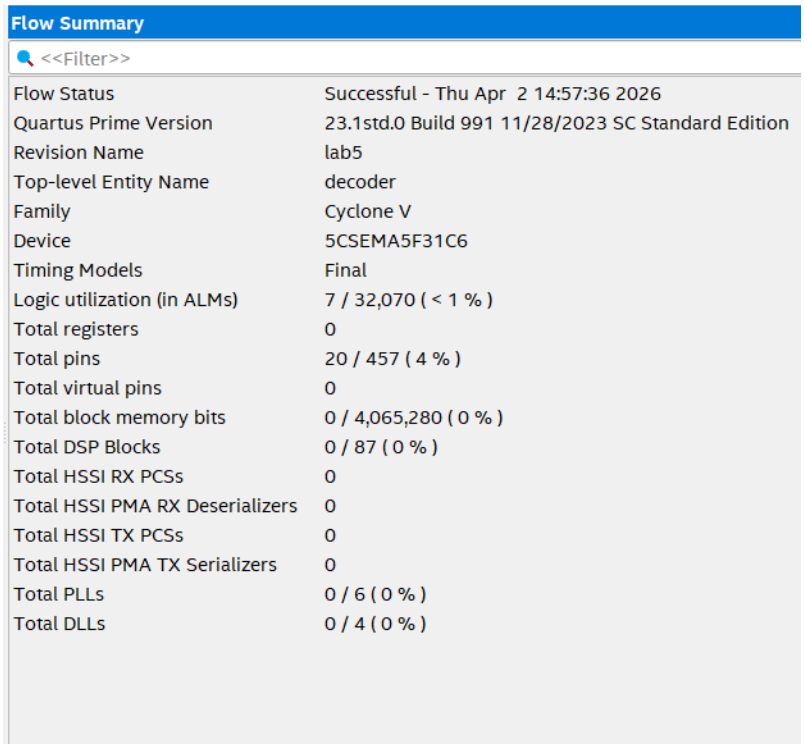


Figure 8. Screenshot of the Decoder module compilation report.

Table 3. Summary for Components for the Decoder Module in PreLab 5.

Feature	Quantity
Total number of logic elements	7
Total number of registers	0
Total number of pins	20
Max number of logic elements that can fit on FPGA	32,070

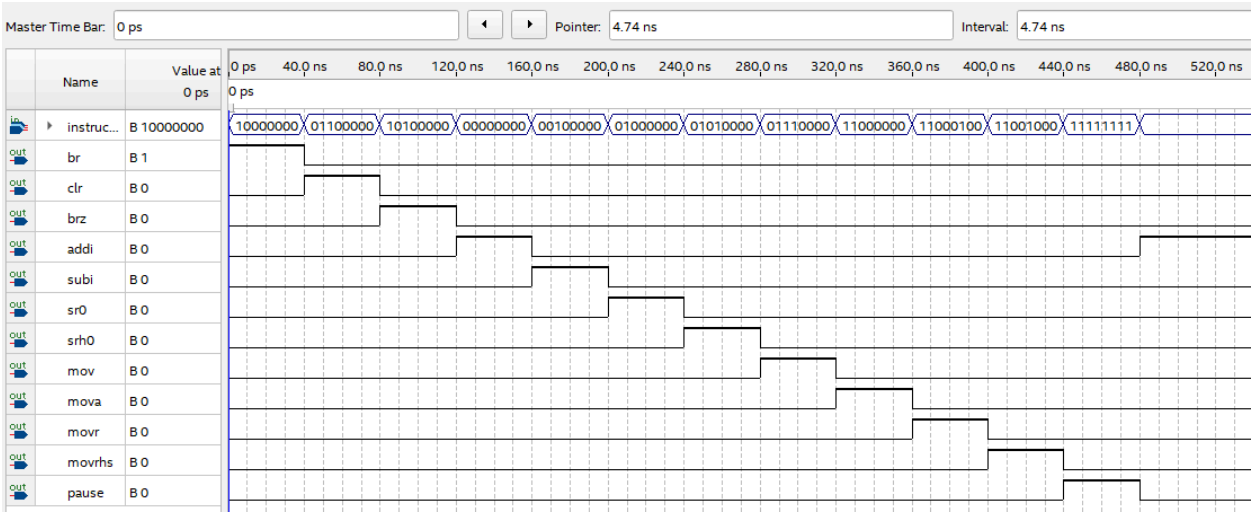


Figure 9. Screenshot of the waveform simulation for the Decoder Module.

```
1  module decoder (input [7:0] instruction,
2                  output br, brz, addi, subi, sr0, srh0, clr, mov, mova, movr, movrhs, pause
3  );
4
5  // input can't produce waveform at 7:2 only 5:0
6
7  assign br = (instruction[7:5] == 3'b100);
8  assign clr = (instruction[7:2] == 6'b011000);
9  assign brz = (instruction[7:5] == 3'b101);
10 assign addi = (instruction[7:5] == 3'b000);
11 assign subi = (instruction[7:5] == 3'b001);
12 assign sr0 = (instruction[7:4] == 4'b0100);
13 assign srh0 = (instruction[7:4] == 4'b0101);
14 assign mov = (instruction[7:4] == 4'b0111);
15 assign mova = (instruction[7:2] == 6'b110000);
16 assign movr = (instruction[7:2] == 6'b110001);
17 assign movrhs = (instruction[7:2] == 6'b110010);
18 assign pause = (instruction[7:0] == 8'b11111111);
19
20
21 endmodule
22
```

Figure 10. Screenshot of the Decoder Module.

Register File

The Register File serves as local storage, consisting of four 8-bit registers labelled R0 to R3. It has two read ports, which increases the performance and allows the ALU to receive two different operands at the same time without requiring additional clock cycles. There is also a synchronous write port to save the results as well. R0 and R1 are used for operations, R2 is used for the motor position, and R3 is used to store the step delay values for the motor.

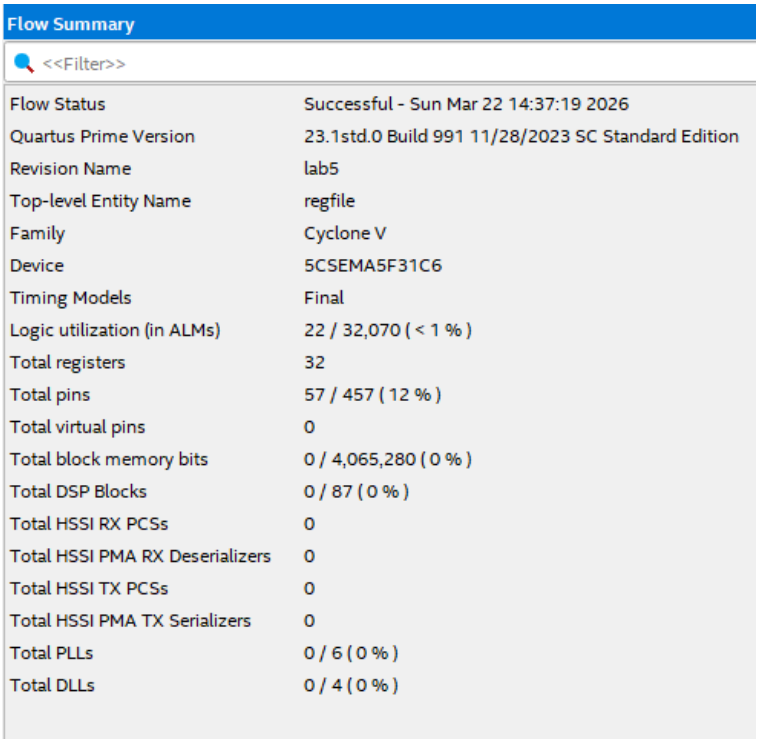


Figure 11. Screenshot of the Register File module compilation report.

Table 4. Summary for Components for the Register File Module in PreLab 5.

Feature	Quantity
Total number of logic elements	22
Total number of registers	32
Total number of pins	57
Max number of logic elements that can fit on FPGA	32,070

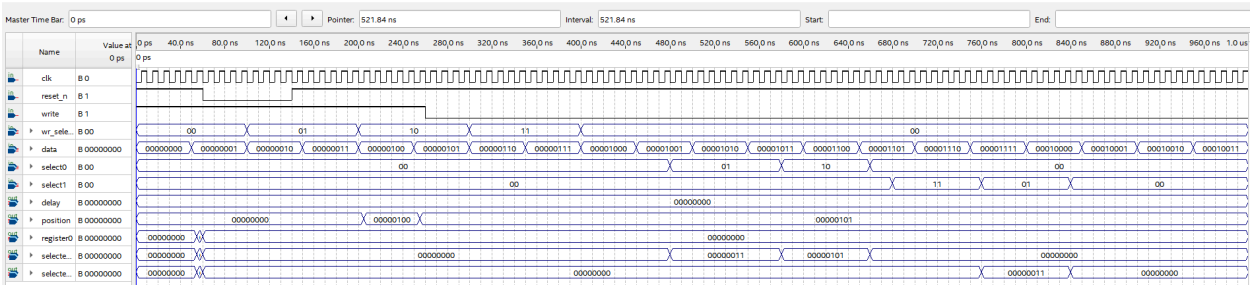


Figure 12. Screenshot of the waveform simulation for the Register File Module.

```
1  module regfile (input clk, reset_n, write, input [7:0] data, input [1:0] select0, select1, wr_select,
2                  output reg [7:0] selected0, selected1, output [7:0] delay, position, register0);
3
4      // The comment /* synthesis preserve */ after the declaration of a register
5      // prevents Quartus from optimizing it, so that it can be observed in simulation
6      // It is important that the comment appear before the semicolon
7      reg [7:0] reg0 /* synthesis preserve */;
8      reg [7:0] reg1 /* synthesis preserve */;
9      reg [7:0] reg2 /* synthesis preserve */;
10     reg [7:0] reg3 /* synthesis preserve */;
11
12     assign register0 = reg0;
13     assign position = reg2;
14     assign delay = reg3;
15
16     always @(posedge clk or negedge reset_n) begin
17        if (!reset_n) begin
18           reg0 <=8'b0;
19           reg1 <=8'b0;
20           reg2 <=8'b0;
21           reg3 <=8'b0;
22        end
23        else if (write) begin
24           case (wr_select)
25              2'b00: reg0 <=data;
26              2'b01: reg1 <=data;
27              2'b10: reg2 <=data;
28              2'b11: reg3 <=data;
29           endcase
30        end
31     end
32
33     always @(*) begin
34        case (select0)
```

Figure 13. Screenshot of the Register File Module.

Write Address Select

The Write Address Select is a multiplexer that manages where data gets saved in the register file. Usually, the first two bits from the instruction (instruction[1:0]) are used to choose the destination register. However, there are specialized motor commands that require the system to force a write to a specific register, such as the position register (R2) or the delay register (R3). The Write Address Select module allows the control unit to override the default instruction fields, ensuring that the state of the motor is updated in the correct location.

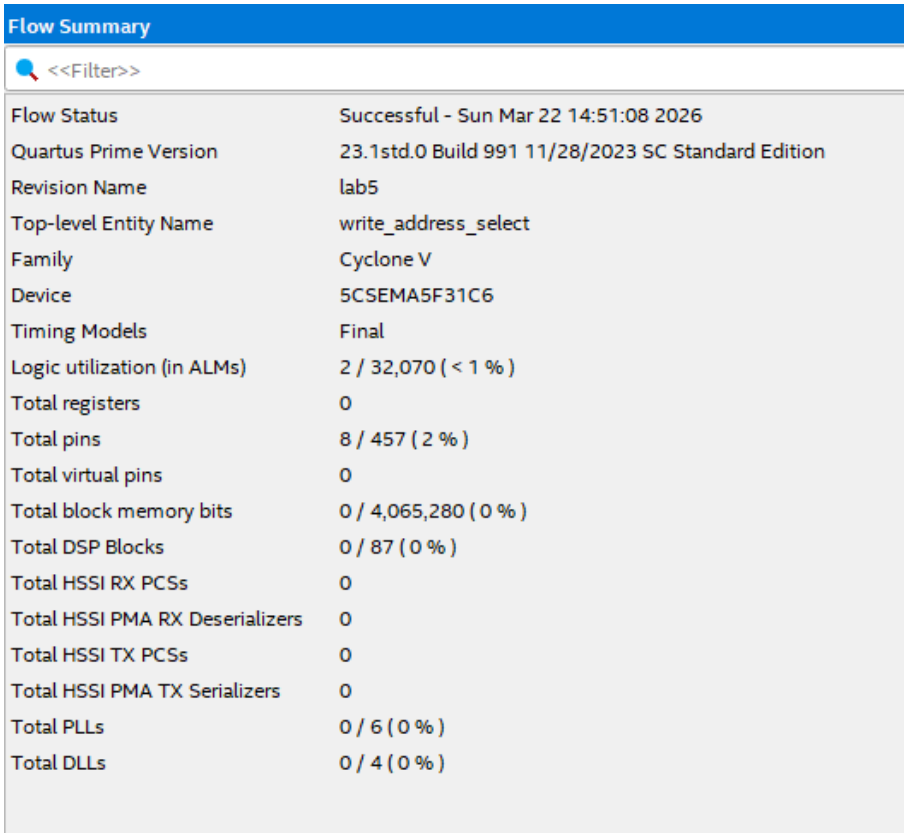


Figure 14. Screenshot of the Write Address Select module compilation report.

Table 5. Summary for Components for the Write Address Select Module in PreLab 5.

Feature	Quantity
Total number of logic elements	2
Total number of registers	0
Total number of pins	8
Max number of logic elements that can fit on FPGA	32,070

```
1   module write_address_select (input [1:0] select, input [1:0] reg_field0, reg_field1, output reg [1:0] write_address);
2
3      always @(*) begin
4         case(select)
5            2'b00: write_address = 2'b0;
6            2'b01: write_address = reg_field0;
7            2'b10: write_address = reg_field1;
8            2'b11: write_address = 2'b10;
9         endcase
10     end
11
12  endmodule
```

Figure 15. Screenshot of the Write Address Select Module.

Result Mux

The Result Mux is a multiplexer which determines the final 8-bit value to be stored in the destination register. Its main purpose is to select between the calculated output of the ALU and a hardwired constant of zero. The hardwired path for the zero value allows the clear (CLR) instruction to be implemented, allowing it to bypass the ALU for reset operations.

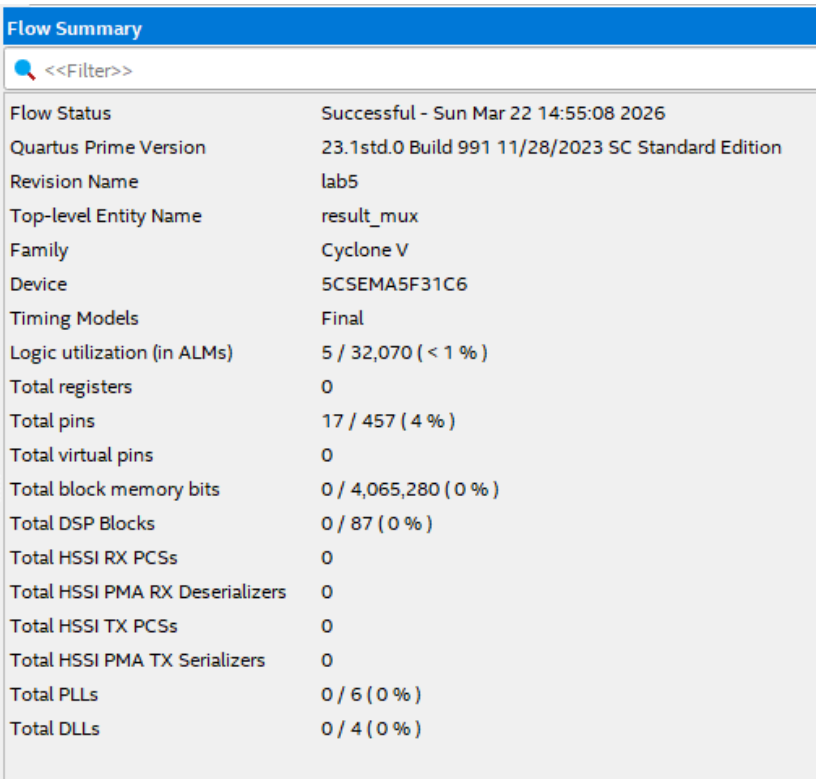


Figure 16. Screenshot of the Result Mux module compilation report.

Table 6. Summary for Components for the Result Mux Module in PreLab 5.

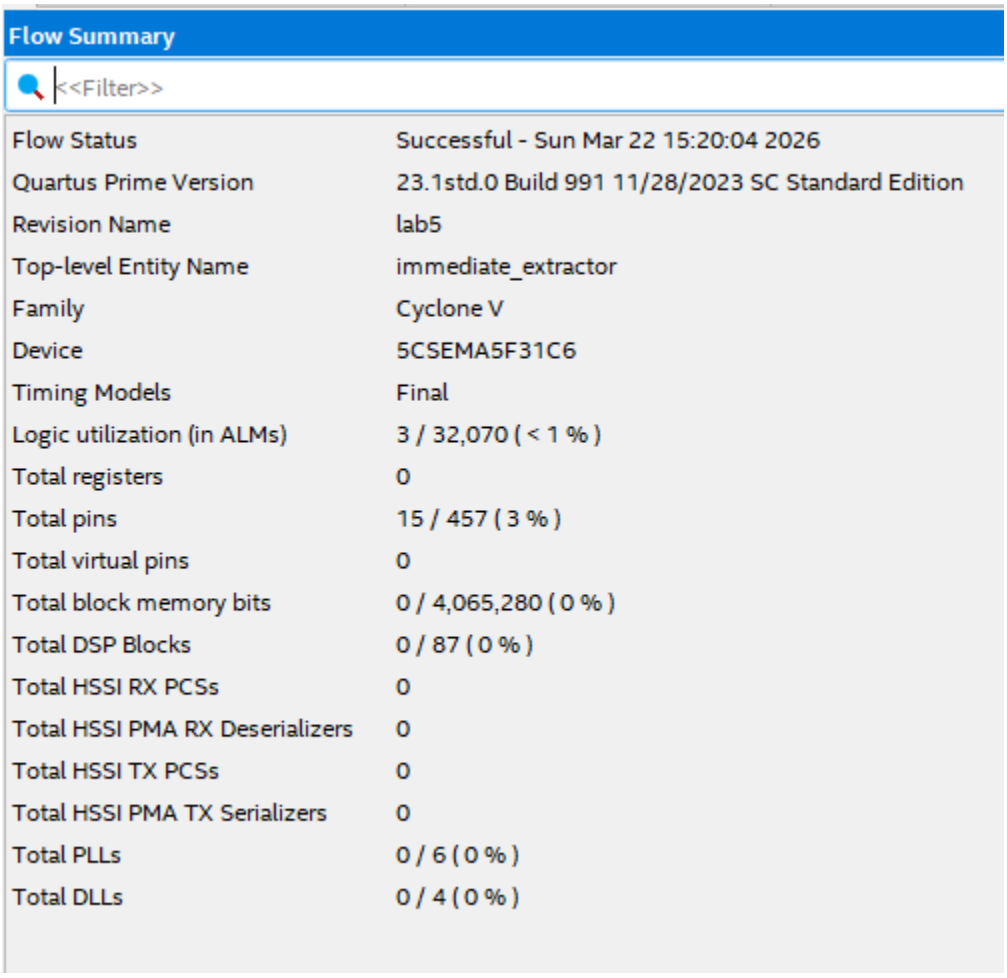
Feature	Quantity
Total number of logic elements	5
Total number of registers	0
Total number of pins	17
Max number of logic elements that can fit on FPGA	32,070

```
1  module result_mux (
2      input select_result,
3      input [7:0] alu_result,
4      output reg [7:0] result
5  );
6
7      always @(*) begin
8          case(select_result)
9              1'b0: result = alu_result;
10             1'b1: result = 8'b0;
11         endcase
12     end
13
14
15
16 endmodule
17
```

Figure 17. Screenshot of the Result Mux Module.

Immediate Extractor

The Immediate Extractor is used to handle constant values or immediates embedded in the instruction machine code. This module extracts the small bit-fields and extends them to a full 8-bit format using zero-padding or sign-extension. This ensures that the ALU receives operands in the correct size, allowing us to use hardcoded values for specific step counts, timing delays, or branch offsets directly within the assembly code.



Flow Summary

<<Filter>>

Flow Status	Successful - Sun Mar 22 15:20:04 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab5
Top-level Entity Name	immediate_extractor
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	3 / 32,070 (< 1 %)
Total registers	0
Total pins	15 / 457 (3 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 18. Screenshot of the Immediate Extractor module compilation report.

Table 7. Summary for Components for the Immediate Extractor Module in PreLab 5.

Feature	Quantity
Total number of logic elements	3
Total number of registers	0
Total number of pins	15
Max number of logic elements that can fit on FPGA	32,070

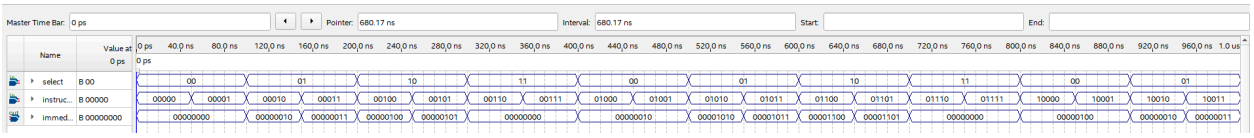


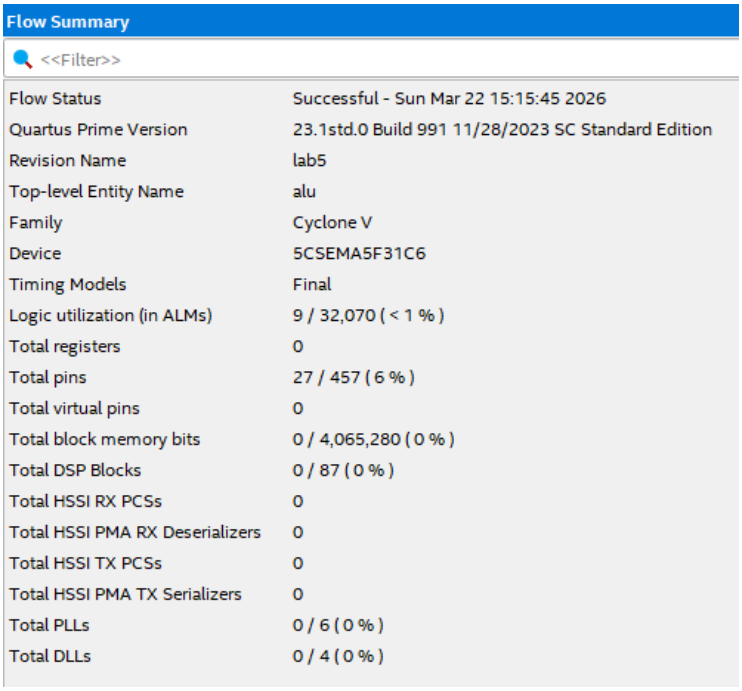
Figure 19. Screenshot of the waveform simulation for the Immediate Extractor Module.

```
1  module immediate_extractor (input [4:0] instruction, input [1:0] select, output reg [7:0] immediate);
2
3      always @(*) begin
4          case (select)
5              2'b00: begin
6                  immediate[7:0] = {5'b00000, instruction[4:2]};
7              end
8              2'b01: begin
9                  immediate[7:0] = {4'b0000, instruction[3:0]};
10             end
11             2'b10: begin
12                 immediate[7:0] = {instruction[4], instruction[4], instruction[4], instruction[4:0]};
13             end
14             2'b11: begin
15                 immediate[7:0] = 8'b0;
16             end
17             default: begin
18                 immediate[7:0] = 8'b0;
19             end
20         endcase
21     end
22
23 endmodule
24
```

Figure 20. Screenshot of the Immediate Extractor Module.

ALU

The ALU is used to perform 8-bit arithmetic and logic operations that are required for motor control. It is capable of addition and subtraction, which are used to update the motor position in R2 or decrement counters. The ALU also handles bit manipulation for instructions like SR0 and SRH0, allowing for the concatenation of 4-bit values. Additionally, it generates the “zero” signal, which is used by the control unit to determine if the branch conditions have been met when the program is executing.



Flow Summary

<<Filter>>

Flow Status	Successful - Sun Mar 22 15:15:45 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab5
Top-level Entity Name	alu
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	9 / 32,070 (< 1 %)
Total registers	0
Total pins	27 / 457 (6 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 21. Screenshot of the ALU module compilation report.

Table 8. Summary for Components for the ALU Module in PreLab 5.

Feature	Quantity
Total number of logic elements	9
Total number of registers	0
Total number of pins	27
Max number of logic elements that can fit on FPGA	32,070

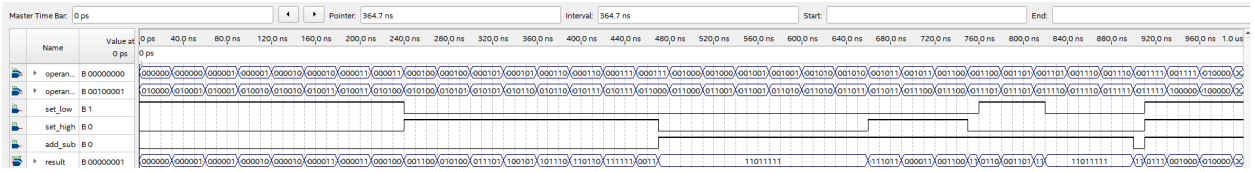


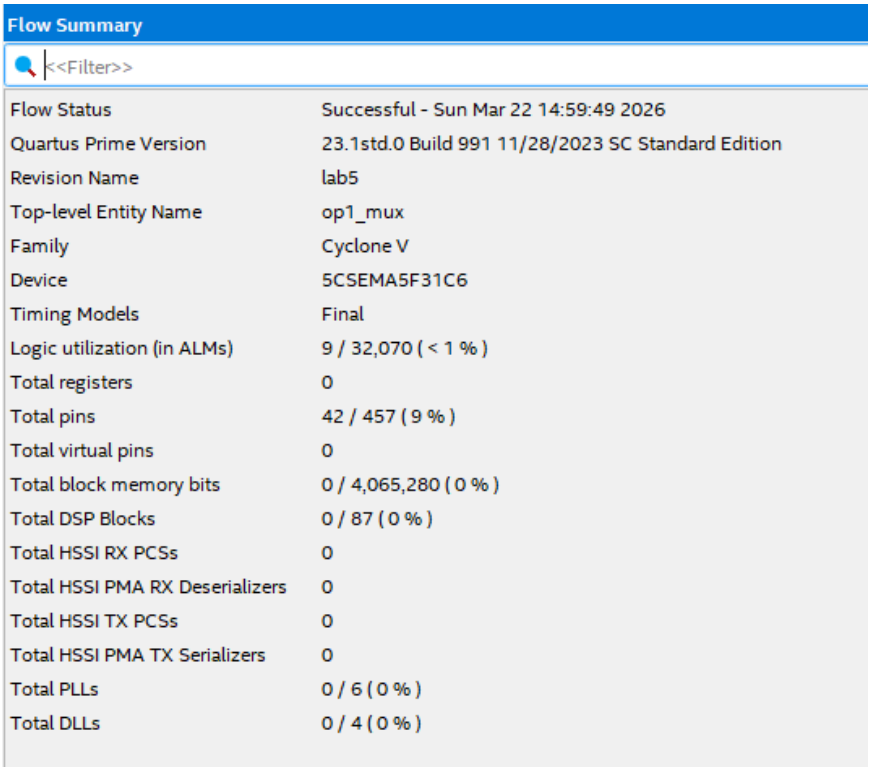
Figure 22. Screenshot of the waveform simulation of the ALU Module.

```
1  module alu (input add_sub, set_low, set_high, input [7:0] operanda , operandb, output reg [7:0] result);
2
3      always @(*) begin
4         result = 8'b0;
5         if (set_low) begin
6            result [7:4] = operanda[7:4];
7            result [3:0] = operandb[3:0];
8         end
9         else if (set_high) begin
10           result[7:4] = operandb[3:0];
11           result[3:0] = operanda[3:0];
12        end
13        else if(add_sub==1'b0) begin
14           result = operanda + operandb;
15        end
16        else if (add_sub==1'b1) begin
17           result = operanda- operandb;
18        end
19     end
20
21
22
23 endmodule
24
```

Figure 23. Screenshot of the ALU Module.

Operand 1 Mux

The Operand 1 Mux controls which value is selected for operanda, which is used by the ALU. The inputs to the mux consist of the program counter, the first read port of the Register File, R2 (position), and R0 (register0). This behaviour allows the ALU to perform a variety of tasks depending on the instruction type.



Flow Summary

<<Filter>>

Flow Status	Successful - Sun Mar 22 14:59:49 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab5
Top-level Entity Name	op1_mux
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	9 / 32,070 (< 1 %)
Total registers	0
Total pins	42 / 457 (9 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 24. Screenshot of the XXX module compilation report.

Table 9. Summary for Components for the Operand 1 Mux Module in PreLab 5.

Feature	Quantity
Total number of logic elements	9
Total number of registers	0
Total number of pins	42
Max number of logic elements that can fit on FPGA	32,070

```
1  module op1_mux (input [1:0] select, input [7:0] pc, register, register0, position,
2                  output reg [7:0] result);
3     always @(*) begin
4        case(select)
5           2'b00: result = pc;
6           2'b01: result = register;
7           2'b10: result = position;
8           2'b11: result = register0;
9        endcase
10    end
11
12
13 endmodule
14
```

Figure 25. Screenshot of the Operand 1 Mux Module.

Operand 2 Mux

The Operand 2 Mux provides the second input to the ALU, selecting between the register file's second read port, the 8-bit output from the Immediate Extractor, or fixed constants (1 or 2). By providing direct access to these small constants, the system can perform tasks, such as incrementing the motor position by a single step or decrementing a loop counter, without using a general-purpose register that stores a fixed value.

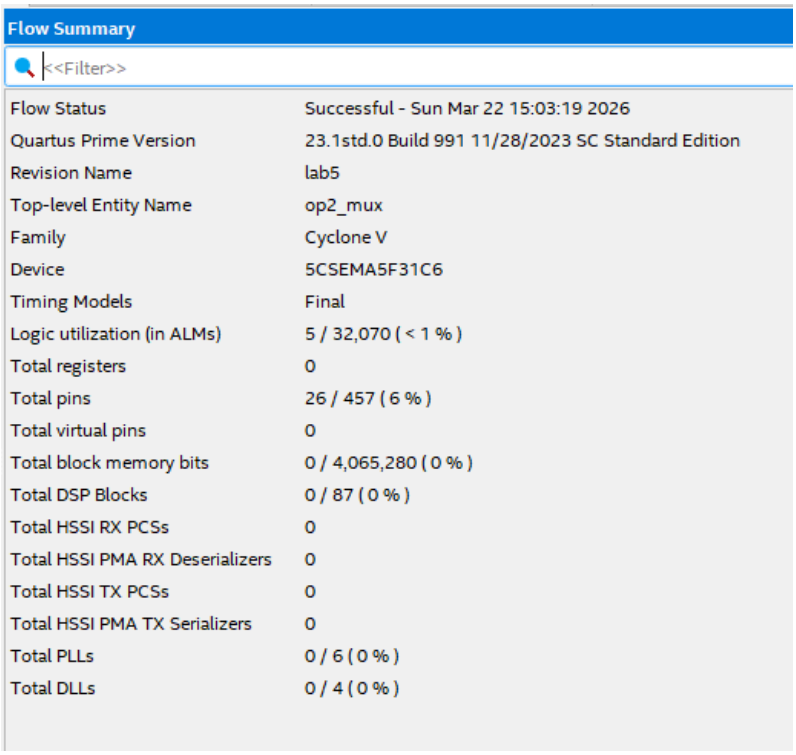


Figure 26. Screenshot of the Operand 2 Mux module compilation report.

Table 10. Summary for Components for the Operand 2 Mux Module in PreLab 5.

Feature	Quantity
Total number of logic elements	5
Total number of registers	0
Total number of pins	26
Max number of logic elements that can fit on FPGA	32,070

```
1  module op2_mux (input [1:0] select, input [7:0] register, immediate,
2                  output reg [7:0] result);
3
4     always@(*) begin
5        case (select)
6           2'b00: result = register;
7           2'b01: result = immediate;
8           2'b10: result = 8'b00000001;
9           2'b11: result = 8'b00000010;
10       endcase
11    end
12
13 endmodule
```

Figure 27. Screenshot of the Operand 2 Mux Module.

Stepper ROM

The Stepper ROM is a specific table that translates the digital position of the motor stored in R2 into the physical electrical signals required by the motor hardware. It uses the three least significant bits of the position register to index a specific 4-bit pattern. This specific pattern is used for half-stepping to ensure that the motor is able to rotate and move properly. The figure below represents the instruction signals for driving the stepper motor to perform the half-steps with – representing a 0 value and + representing 1. The conversion of the instruction signals are shown in the stepper_rom.mif in order.

Location	b_3	b_2	b_1	b_0
000	–	–	+	–
001	–	+	+	–
010	–	+	–	–
011	–	+	–	+
100	–	–	–	+
101	+	–	–	+
110	+	–	–	–
111	+	–	+	–

Figure 28. Instruction signals for driving the stepper motor to perform half-stepping used in stepper_rom.mif

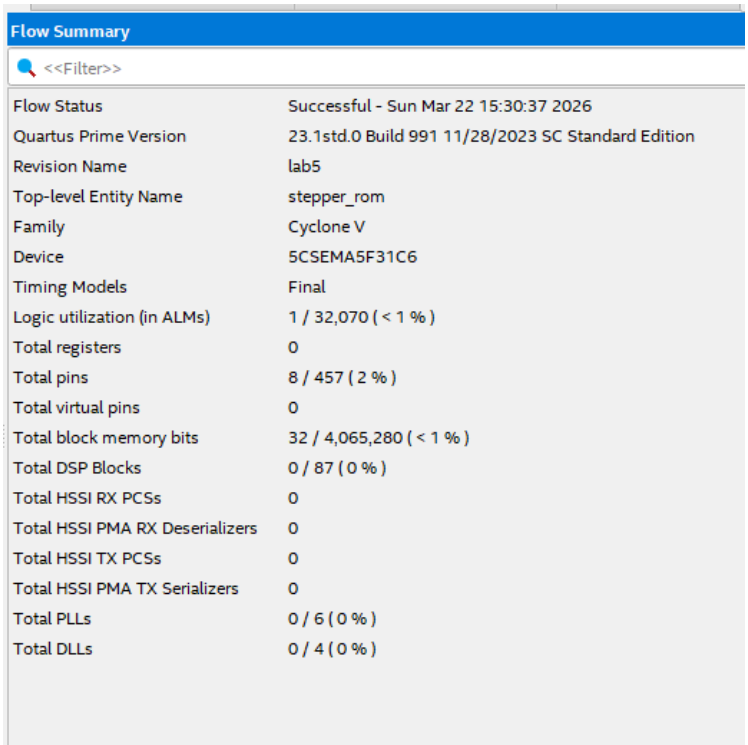
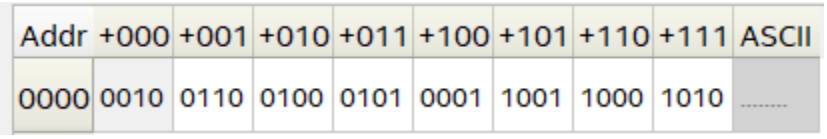


Figure 29. Screenshot of the Operand 2 Mux module compilation report.

Table 11. Summary for Components for the Operand 2 Mux Module in PreLab 5.

Feature	Quantity
Total number of logic elements	1
Total number of registers	0
Total number of pins	8
Max number of logic elements that can fit on FPGA	32,070



Addr	+000	+001	+010	+011	+100	+101	+110	+111	ASCII
0000	0010	0110	0100	0101	0001	1001	1000	1010	.....

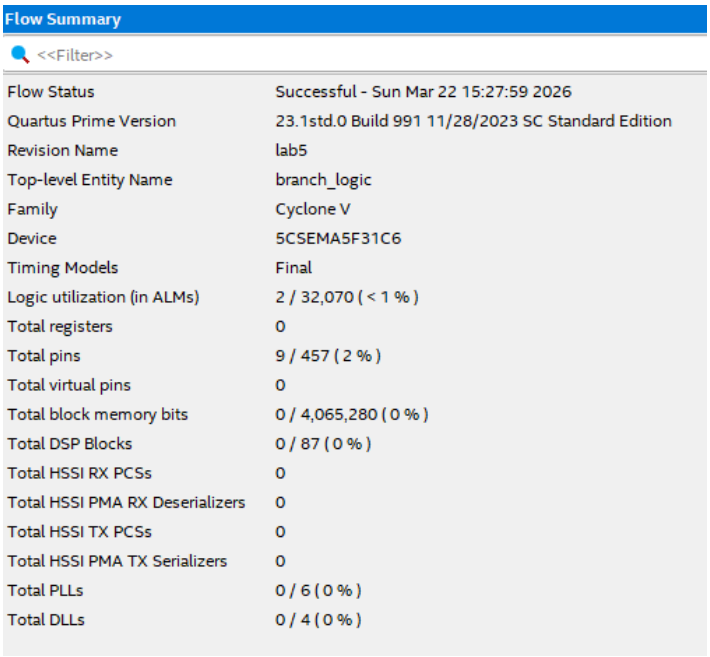
Figure 30. Screenshot of stepper_rom.mif

```
40  module stepper_rom (
41     address,
42     clock,
43     q);
44
45     input [2:0]  address;
46     input   clock;
47     output   [3:0]  q;
48  `ifndef ALTERA_RESERVED_QIS
49  // synopsys translate_off
50  `endif
51     tri1    clock;
52  `ifndef ALTERA_RESERVED_QIS
53  // synopsys translate_on
54  `endif
55
56     wire [3:0] sub_wire0;
57     wire [3:0] q = sub_wire0[3:0];
58
59     altsyncram  altsyncram_component (
60               .address_a (address),
61               .clock0 (clock),
62               .q_a (sub_wire0),
63               .aclr0 (1'b0),
64               .aclr1 (1'b0),
65               .address_b (1'b1),
66               .addressstall_a (1'b0),
67               .addressstall_b (1'b0),
68               .byteena_a (1'b1),
69               .byteena_b (1'b1),
70               .clock1 (1'b1),
71               .clocken0 (1'b1),
72               .clocken1 (1'b1),
73               .clocken2 (1'b1),
```

Figure 31. Screenshot of the Operand 2 Mux Module.

Branch Logic

The Branch Logic is a combinational module that monitors R0. It checks if the value currently stored in Register 0 is exactly zero and outputs a corresponding signal to the Control Unit. The status bit is used for the BRZ instruction, allowing the processor to evaluate logic conditions, such as determining if a loop has completed its iterations and then signals the Program Counter to either continue sequentially or jump to a new target address.



Flow Summary

<<Filter>>

Flow Status	Successful - Sun Mar 22 15:27:59 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab5
Top-level Entity Name	branch_logic
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	2 / 32,070 (< 1 %)
Total registers	0
Total pins	9 / 457 (2 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 32. Screenshot of the Branch Logic module compilation report.

Table 12. Summary for Components for the Branch Logic Module in PreLab 5.

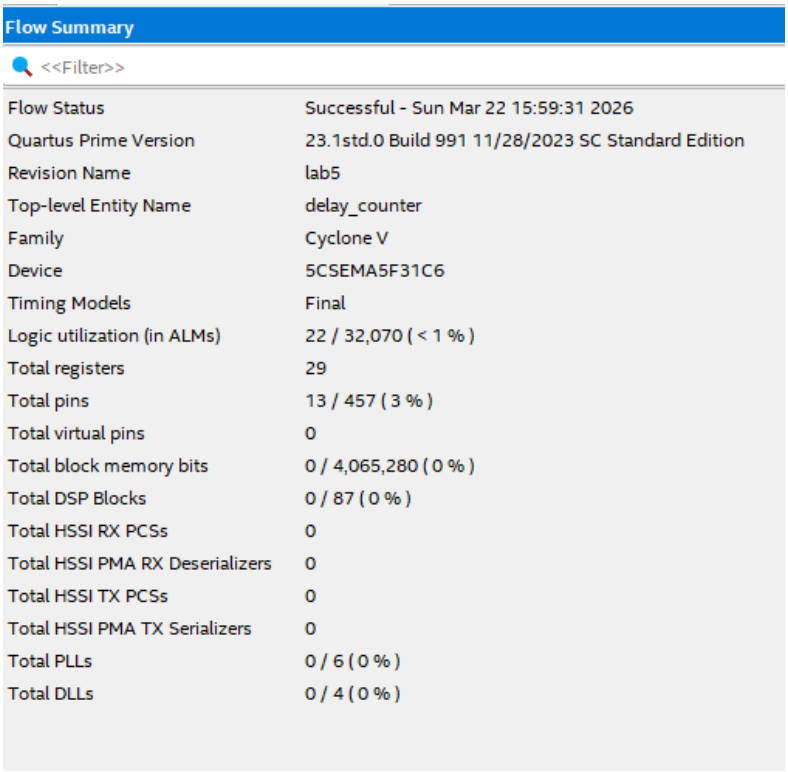
Feature	Quantity
Total number of logic elements	2
Total number of registers	0
Total number of pins	9
Max number of logic elements that can fit on FPGA	32,070

```
1   module branch_logic (input [7:0] register0, output reg branch);
2   //had to change branch to output reg instead of just output for combinational logic
3
4      always @(*) begin
5         if (register0==8'b0) begin
6            branch = 1'b1;
7         end
8         else begin
9            branch = 1'b0;
10        end
11     end
12
13  endmodule
```

Figure 33. Screenshot of the Branch Logic Module.

Delay Counter

The Delay Counter is a module designed to manage the time interval between motor steps, preventing the motor from moving faster than the physical hardware can respond. It uses the 50MHz system clock and the value stored in R3 to create time delays. Once it is activated by the control unit, it counts down in the background and asserts a "done" signal when it is completed. This allows the ASIP to maintain consistent, programmable motor speeds.



Flow Summary

<<Filter>>

Flow Status	Successful - Sun Mar 22 15:59:31 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab5
Top-level Entity Name	delay_counter
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	22 / 32,070 (< 1 %)
Total registers	29
Total pins	13 / 457 (3 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 34. Screenshot of the Delay Counter module compilation report.

Table 13. Summary for Components for the Delay Counter Module in PreLab 5.

Feature	Quantity
Total number of logic elements	22
Total number of registers	29
Total number of pins	13
Max number of logic elements that can fit on FPGA	32,070

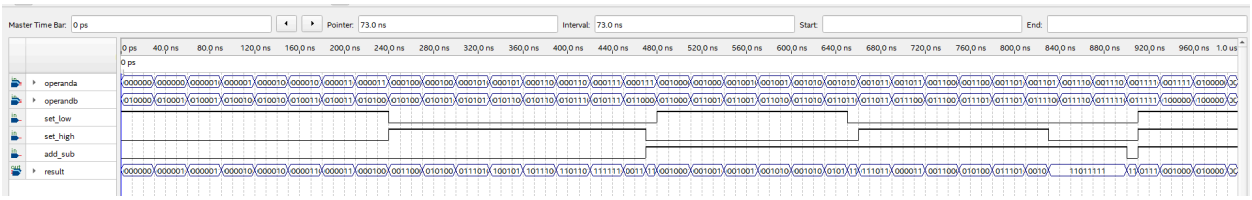


Figure 35. Screenshot of the waveform simulation of the Delay Counter Module.

```
module delay_counter (input clk, reset_n, start, enable, input [7:0] delay, output reg done);
parameter BASIC_PERIOD=20'd500000;    // can change this value to make delay longer

    reg [19:0] count_cycle;
    reg [7:0] count_delay;

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            count_cycle <= 20'b0;
            count_delay <= 8'b0;
            done <= 1'b0;
        end
        else if (start) begin
            count_cycle <= 20'b0;
            count_delay <= delay;
            done <= 1'b0;
        end

        else if (enable) begin
            if (count_cycle < BASIC_PERIOD - 1) begin
                count_cycle <= count_cycle + 1;
            end
            else begin
                count_cycle <= 20'b0;
                if (count_delay > 0) begin
                    count_delay <= count_delay - 1;
                end
                if (count_delay == 1) begin
                    done <= 1;
                end
                else begin
                    done <= 0;
                end
            end
```

Figure 36. Screenshot of the Delay Counter Module.

TEMP Register

The TEMP register is a special internal register that is used for motor movement instructions like MOVR and MOVRHS. The main function of this register is to hold a count of the number of steps or half-steps that a motor has to make in executing a particular instruction. When a motor movement instruction is initiated, a count of steps or half-steps to be made by a motor is stored in this register. The register acts as a counter that increments or decrements its value depending on whether it is a positive or negative operation. Additionally, it has status signals that indicate whether it holds a positive, negative, or zero value, which are used to determine whether a motor movement operation has been completed or not.

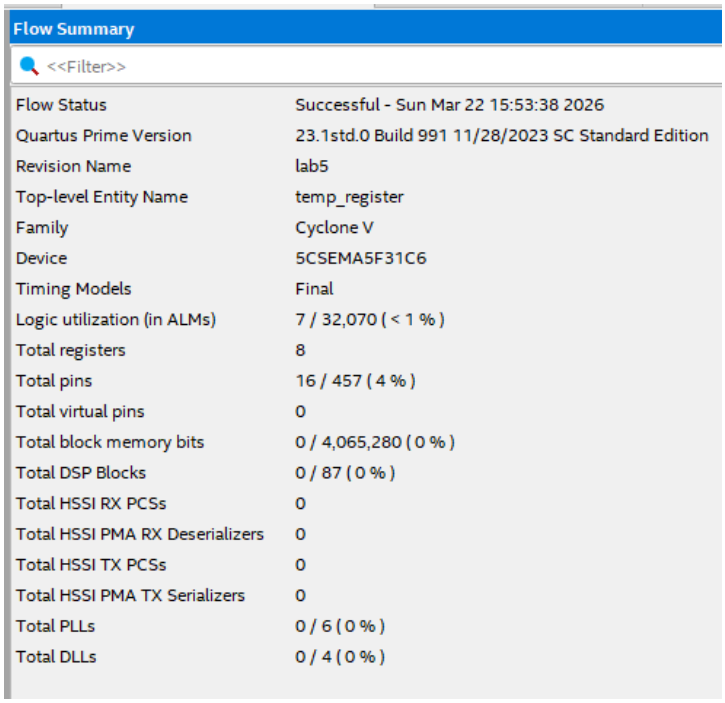


Figure 37. Screenshot of the TEMP Register module compilation report.

Table 14. Summary for Components for the TEMP Register Module in PreLab 5.

Feature	Quantity
Total number of logic elements	7
Total number of registers	8
Total number of pins	16
Max number of logic elements that can fit on FPGA	32,070

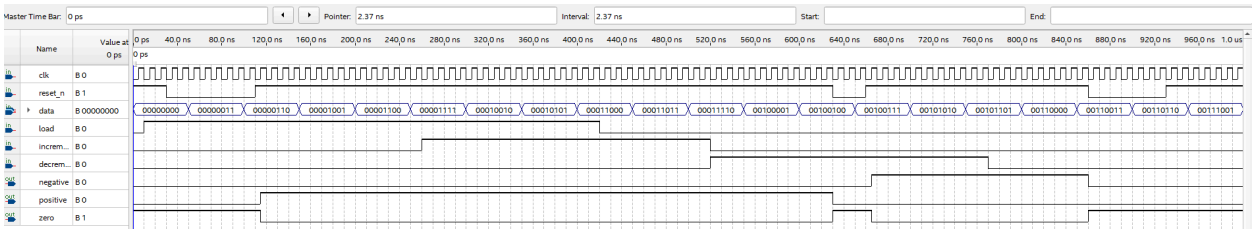


Figure 38. Screenshot of the waveform simulation of the TEMP Register Module.

```
1  module temp_register (input clk, reset_n, load, increment, decrement, input signed [7:0] data,
2                output negative, positive, zero); // changed data to signed to make it simpler and the code work
3
4     reg signed [7:0] halfsteps; //signed since temp can be negative or positive
5
6     assign positive = (halfsteps>0);
7     assign zero =  (halfsteps==0);
8     assign negative = (halfsteps<0);
9
10    always @(posedge clk or negedge reset_n) begin
11       if (!reset_n) begin
12          halfsteps <= 0;
13       end
14       else if(load) begin
15          halfsteps <= data;
16       end
17       else if(increment) begin
18          halfsteps <=halfsteps +1;
19       end
20       else if(decrement) begin
21          halfsteps <=halfsteps-1;
22       end
23    end
24
25 endmodule
```

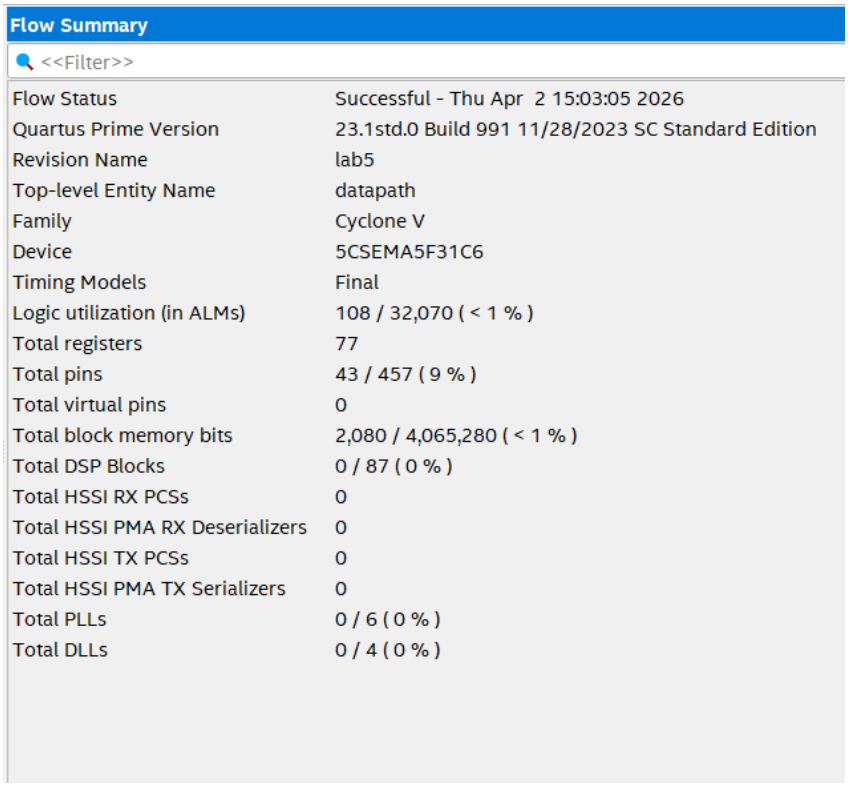
Figure 39. Screenshot of the TEMP Register Module.

Datapath

The datapath of the ASIP consists of several interconnected modules. These modules are used for the execution of the instructions as well as motor control. The program counter and the instruction memory are used for the control of the program. The register file is used for the storage of the general-purpose registers as well as the special-purpose registers. The ALU is used for the execution of arithmetic and logical operations. The multiplexers are used for the selection of the operands as well as the result. Additional modules, including the immediate extractor, branch logic, and delay counter, support instruction decoding, control flow, and precise timing. Together, these components form a structured datapath that processes instructions.

There are several different waveforms shown below to prove the datapath.v is functional. An important note for verification is if one instruction is set to high, all the other instructions are set to low. If BRZ is high, the other instructions are set to low.

The top-level module for tutorial 5 is datapath.v.



Flow Summary

<<Filter>>

Flow Status	Successful - Thu Apr 2 15:03:05 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab5
Top-level Entity Name	datapath
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	108 / 32,070 (< 1 %)
Total registers	77
Total pins	43 / 457 (9 %)
Total virtual pins	0
Total block memory bits	2,080 / 4,065,280 (< 1 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 40. Screenshot of the Top Level module compilation report.

Table 15. Summary for Components for the Top Level Module in PreLab 5.

Feature	Quantity
Total number of logic elements	108
Total number of registers	77
Total number of pins	43
Max number of logic elements that can fit on FPGA	32,070

The BRZ logic below works since whenever BRZ is high, ADDI and all other instructions are set to low. BRZ is set to high whenever reset is set to low since reset sets instruction 0 as the current logic. And since BRZ in this case was set to instruction 0, BRZ is high.

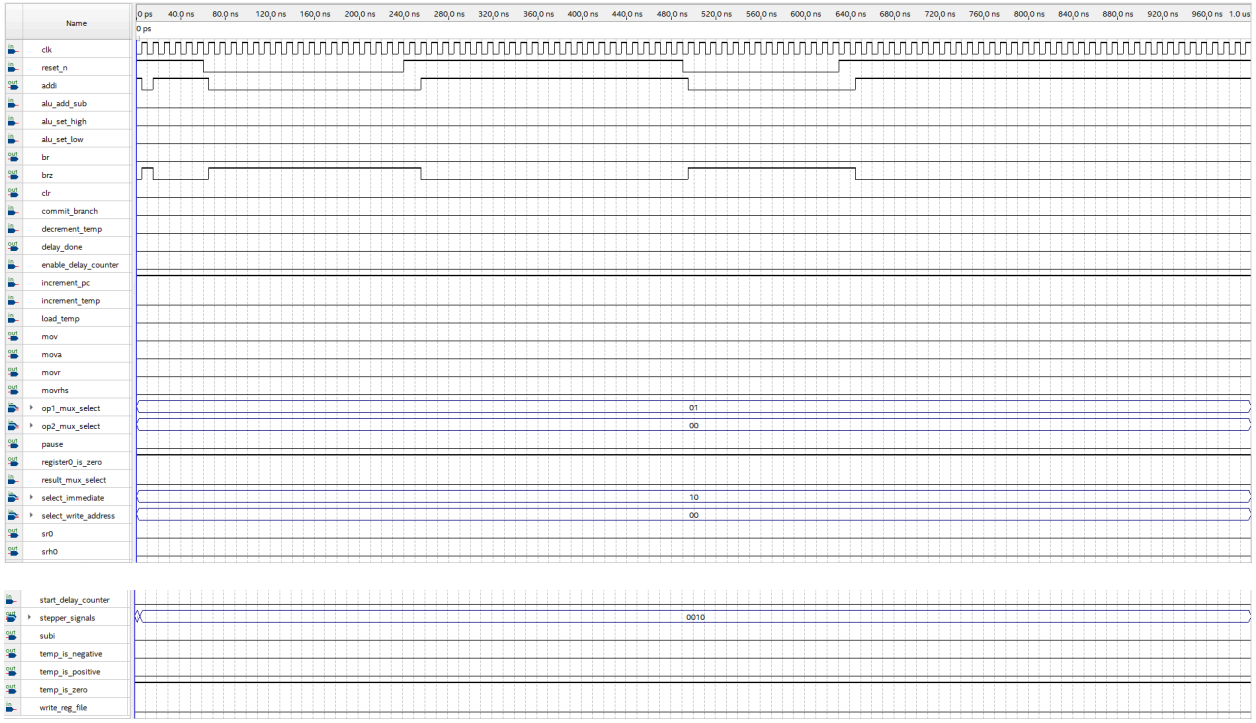


Figure 41. BRZ logic

ADDI below is set to high throughout the whole thing with the correct corresponding ADDI input parameters.

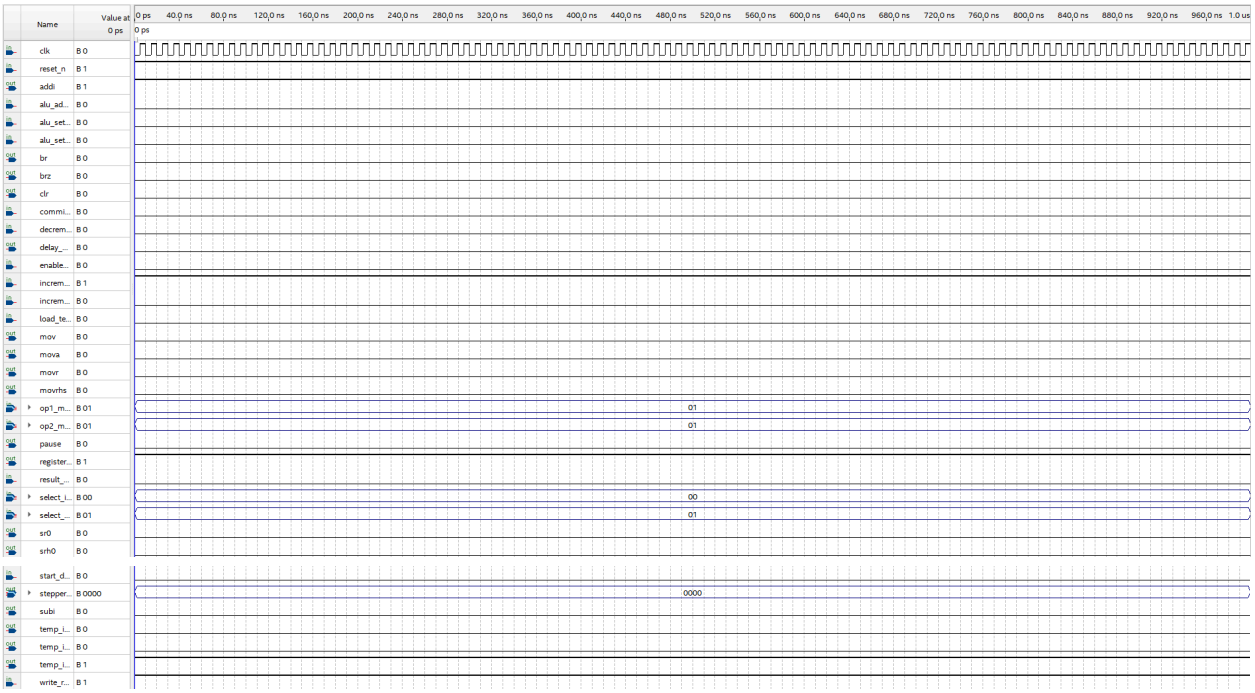


Figure 42. ADDI logic.

MOVR logic is proven true since the set of instrucion_rom.mif was set to MOVR for the majority of the beginning portion with different values of MOVR. This indicates the correct instruction values are run.

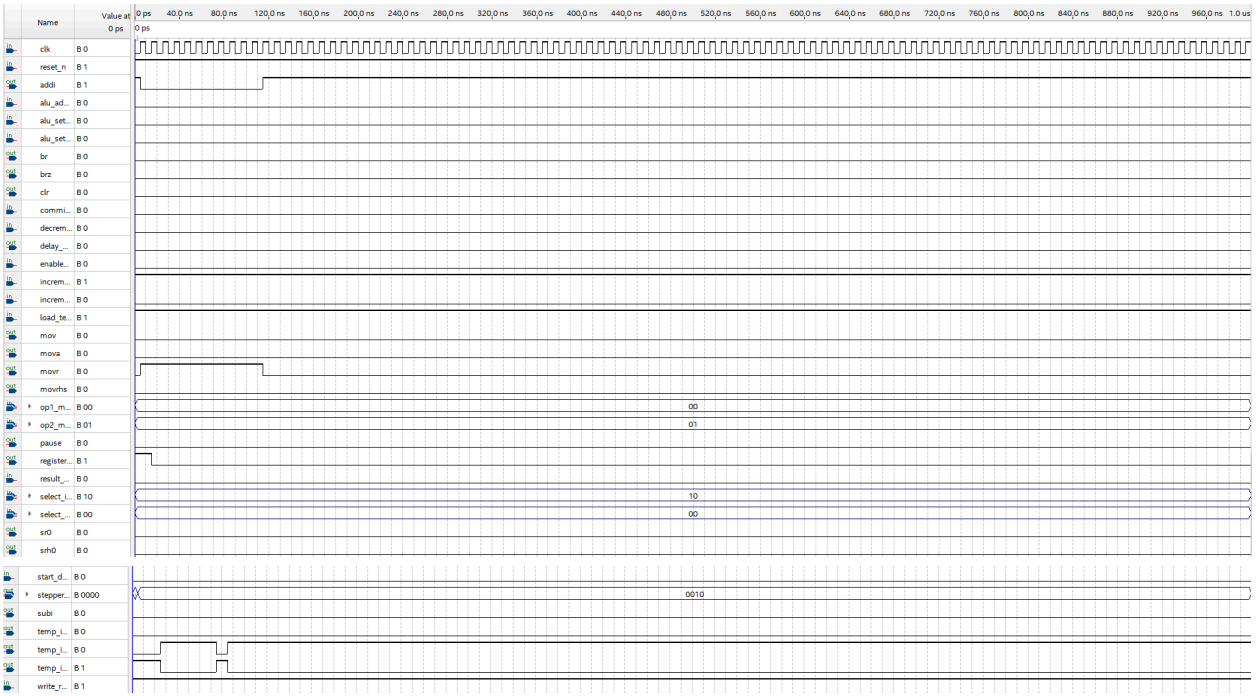


Figure 43. MOVR logic.

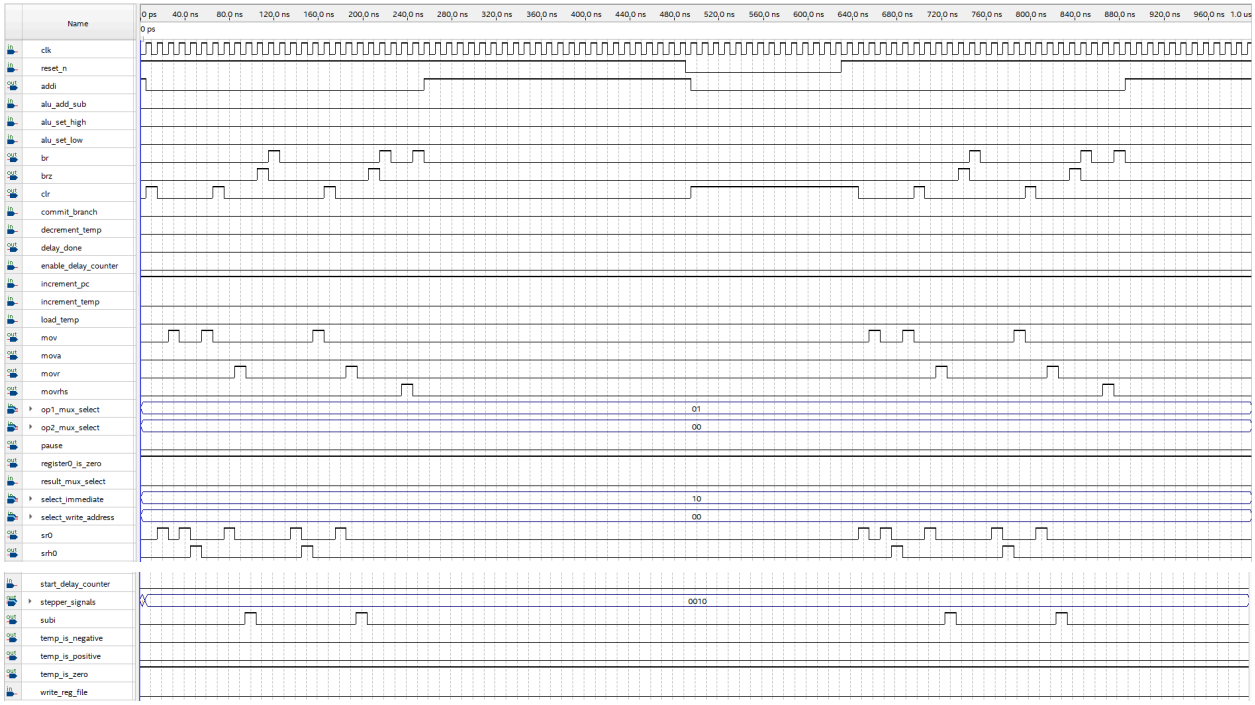


Figure 44. Instruction_rom.mif shown in the Instruction Module section waveform with all signals (MOVR, BRZ, BR, SUBI, etc).

Lastly, the figure showing the instruction module section waveform uses the Spin3.mif, which demonstrates each instruction that eventually results in the motor spinning 3 steps clockwise, 4 half-steps counter-clockwise, with a ½ second delay between each step.

```
module datapath (input clk, reset_n,
                 // Control signals
                 input write_reg_file, result_mux_select,
                 input [1:0] op1_mux_select, op2_mux_select,
                 input start_delay_counter, enable_delay_counter,
                 input commit_branch, increment_pc,
                 input alu_add_sub, alu_set_low, alu_set_high,
                 input load_temp, increment_temp, decrement_temp,
                 input [1:0] select_immediate,
                 input [1:0] select_write_address,
                 // Status outputs
                 output br, brz, addi, subi, sr0, srh0, clr, mov, mova, movr, movrhs, pause,
                 output delay_done,
                 output temp_is_positive, temp_is_negative, temp_is_zero,
                 output register0_is_zero,
                 // Motor control outputs
                 output [3:0] stepper_signals
);
 // The comment /*synthesis keep*/ after the declaration of a wire
 // prevents Quartus from optimizing it, so that it can be observed in simulation
 // It is important that the comment appear before the semicolon
 wire [7:0] position /*synthesis keep*/;
 wire [7:0] delay /*synthesis keep*/;
 wire [7:0] register0 /*synthesis keep*/;

 // Based on our values labeled in Appendix B
 wire [7:0] instruction_out /*synthesis keep*/;
 wire [7:0] PC_out /*synthesis keep*/;
 wire [7:0] ALU_out /*synthesis keep*/;
 wire [7:0] sel0_out /*synthesis keep*/;
 wire [7:0] sel1_out /*synthesis keep*/;
 wire [7:0] immed_op_out /*synthesis keep*/;
 wire [7:0] muxres_out /*synthesis keep*/;
 wire [7:0] mux1_out /*synthesis keep*/;
```

Figure 45. Screenshot of the module of the Top Level Module.

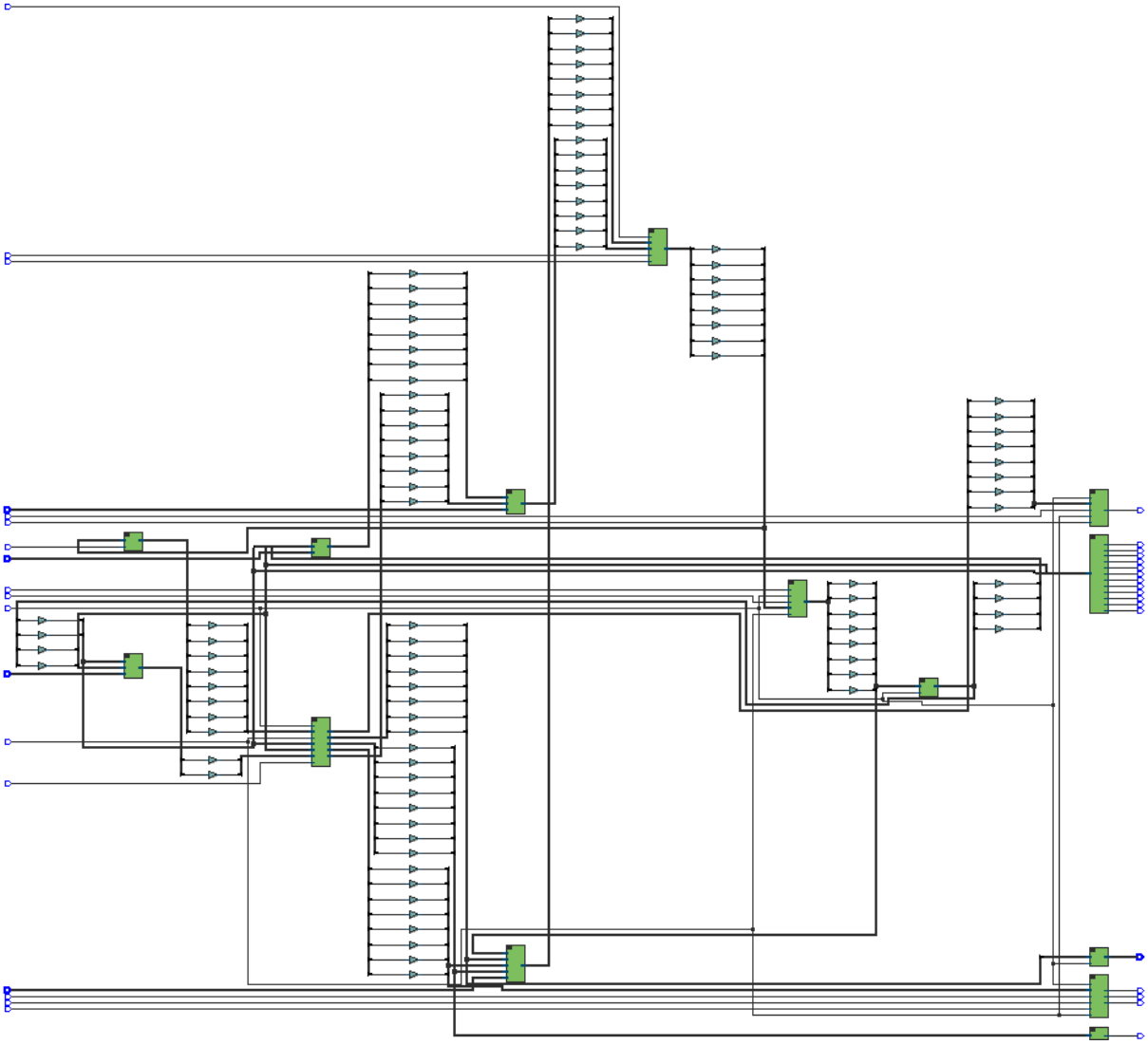


Figure 46. Screenshot of RTL view for datapath.v.

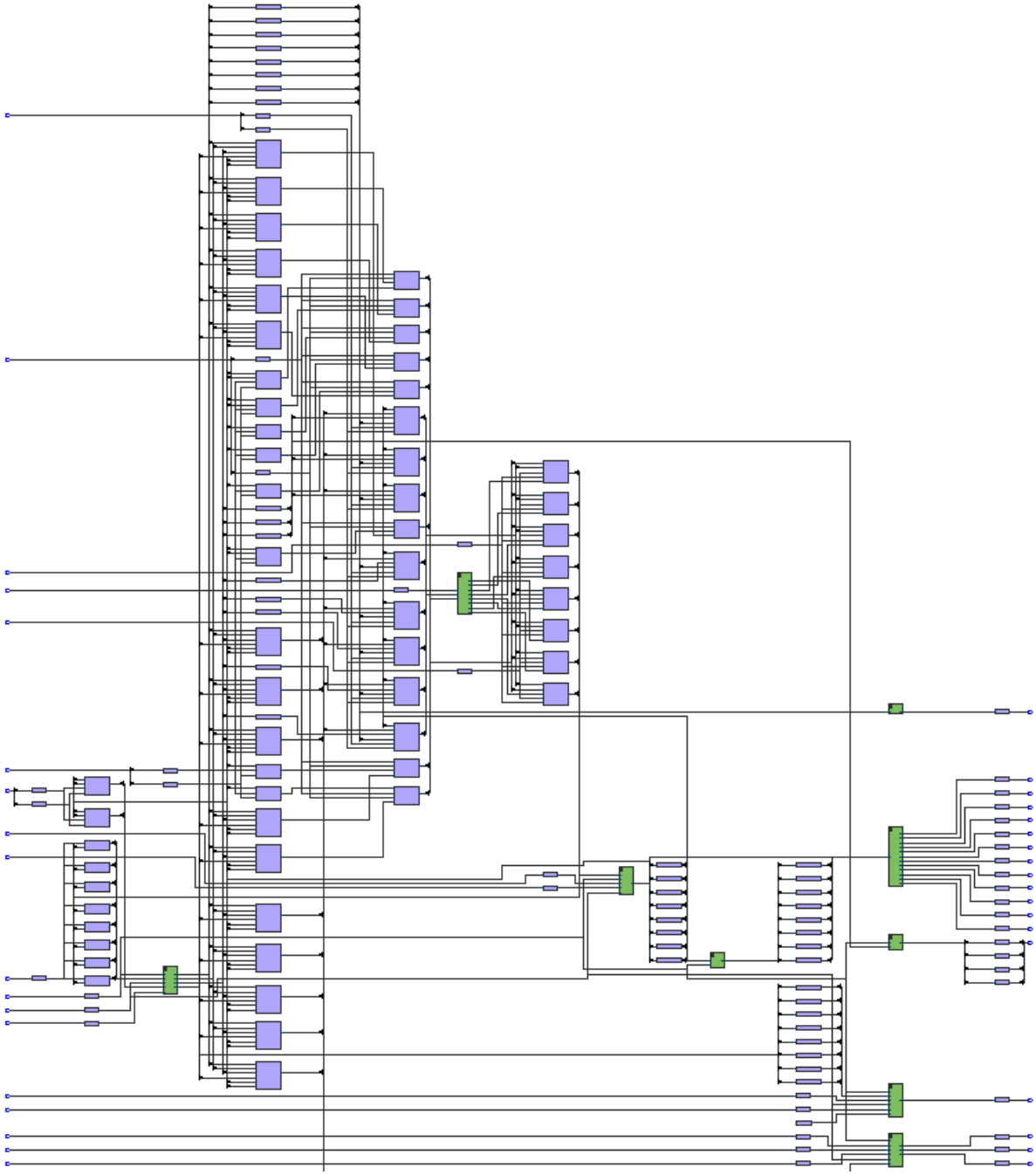


Figure 47. Screenshot of Post-Mapping view for datapath.v.

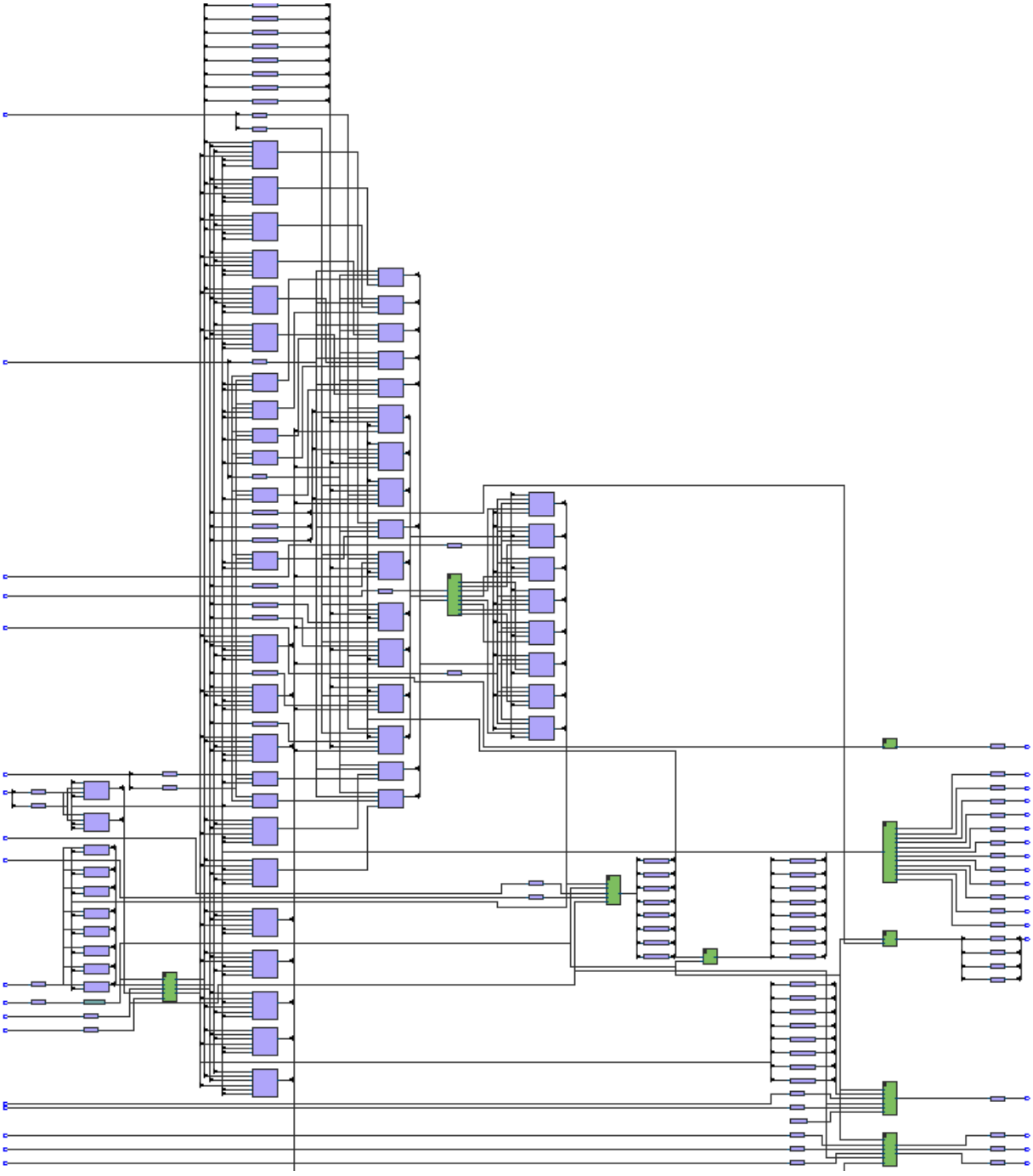


Figure 48. Screenshot of Post-Fitting view for datapath.v.

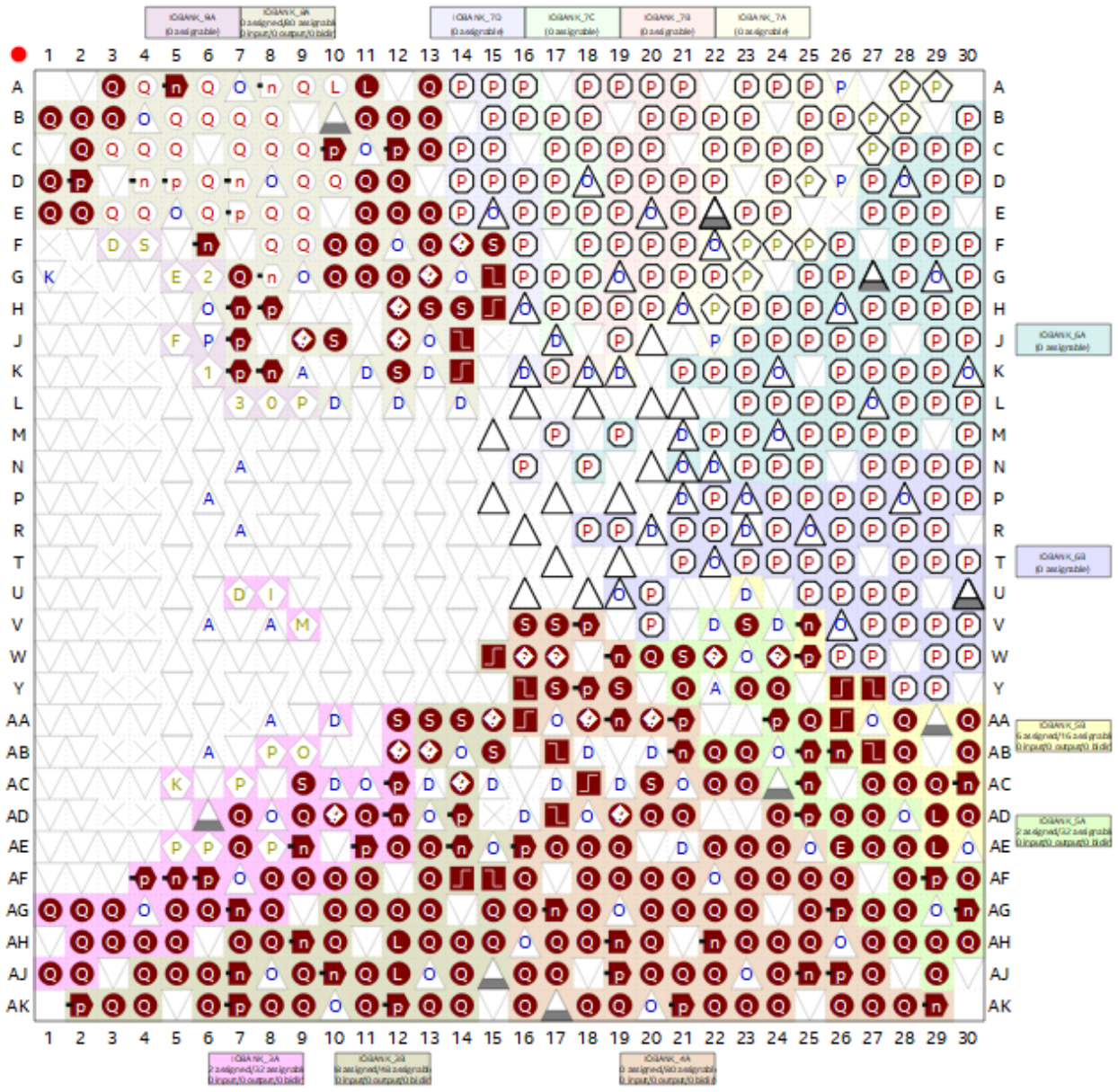


Figure 49a. Screenshot of Pin Planner view for datapath.v.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate
addi	Output				PIN_AK14	2.5 V (default)		12mA (default)	1 (default)
alu_add_sub	Input				PIN_AJ20	2.5 V (default)		12mA (default)	
alu_set_high	Input				PIN_AH18	2.5 V (default)		12mA (default)	
alu_set_low	Input				PIN_AH17	2.5 V (default)		12mA (default)	
br	Output				PIN_W15	2.5 V (default)		12mA (default)	1 (default)
brz	Output				PIN_W16	2.5 V (default)		12mA (default)	1 (default)
clk	Input				PIN_AB27	2.5 V (default)		12mA (default)	
clr	Output				PIN_AG17	2.5 V (default)		12mA (default)	1 (default)
commit_branch	Input				PIN_AG16	2.5 V (default)		12mA (default)	
decrement_temp	Input				PIN_AF19	2.5 V (default)		12mA (default)	
delay_done	Output				PIN_AE17	2.5 V (default)		12mA (default)	1 (default)
enable_delay_counter	Input				PIN_AH15	2.5 V (default)		12mA (default)	
increment_pc	Input				PIN_AF18	2.5 V (default)		12mA (default)	
increment_temp	Input				PIN_AJ21	2.5 V (default)		12mA (default)	
load_temp	Input				PIN_AG20	2.5 V (default)		12mA (default)	
mov	Output				PIN_W17	2.5 V (default)		12mA (default)	1 (default)
mova	Output				PIN_AJ19	2.5 V (default)		12mA (default)	1 (default)
movr	Output				PIN_AH12	2.5 V (default)		12mA (default)	1 (default)
movrhs	Output				PIN_AF16	2.5 V (default)		12mA (default)	1 (default)
op1_mux_select[1]	Input				PIN_AJ17	2.5 V (default)		12mA (default)	
op1_mux_select[0]	Input				PIN_AB17	2.5 V (default)		12mA (default)	
op2_mux_select[1]	Input				PIN_AK16	2.5 V (default)		12mA (default)	
op2_mux_select[0]	Input				PIN_AG18	2.5 V (default)		12mA (default)	
pause	Output				PIN_AG22	2.5 V (default)		12mA (default)	1 (default)
register0_is_zero	Output				PIN_AG21	2.5 V (default)		12mA (default)	1 (default)
reset_n	Input				PIN_AE16	2.5 V (default)		12mA (default)	
result_mux_select	Input				PIN_AA16	2.5 V (default)		12mA (default)	
select_immediate[1]	Input				PIN_AK19	2.5 V (default)		12mA (default)	
select_immediate[0]	Input				PIN_AJ16	2.5 V (default)		12mA (default)	
select_wri...address[1]	Input				PIN_AH19	2.5 V (default)		12mA (default)	
select_wri...address[0]	Input				PIN_AH20	2.5 V (default)		12mA (default)	
sr0	Output				PIN_AK18	2.5 V (default)		12mA (default)	1 (default)
srh0	Output				PIN_V17	2.5 V (default)		12mA (default)	1 (default)
start_delay_counter	Input				PIN_Y16	2.5 V (default)		12mA (default)	
stepper_signals[3]	Output				PIN_AE18	2.5 V (default)		12mA (default)	1 (default)
stepper_signals[2]	Output				PIN_AD17	2.5 V (default)		12mA (default)	1 (default)
stepper_signals[1]	Output				PIN_AH24	2.5 V (default)		12mA (default)	1 (default)
stepper_signals[0]	Output				PIN_AC18	2.5 V (default)		12mA (default)	1 (default)
subi	Output				PIN_AJ14	2.5 V (default)		12mA (default)	1 (default)
temp_is_negative	Output				PIN_AE19	2.5 V (default)		12mA (default)	1 (default)
temp_is_positive	Output				PIN_AH22	2.5 V (default)		12mA (default)	1 (default)
temp_is_zero	Output				PIN_AG23	2.5 V (default)		12mA (default)	1 (default)
write_reg_file	Input				PIN_V16	2.5 V (default)		12mA (default)	

Figure 49b. Screenshot of Pin Planner pins for datapath.v.

Lab 5 Prelab Report

Overview

In this prelab, assembly programs were written in order to perform different functions using the custom ASIP instruction set or prompts provided. These programs were then converted into machine code for implementation in the instruction memory. Two assembly programs were created. The studentNumber.txt verifies arithmetic and register operations, while the second program for Spin3.txt is used to test motor control instructions with stepping, direction control, delay, and looping behaviour.

The assembly language utilizes instructions from the ASIP instruction sets and their provided usage are shown below:

- CLR: to initialize registers and change the values in the registers.
- SR0 and SRH0: to construct 8-bit values with SR0 set for the lower 4-bit (8, 4, 2, 1)and SRH0 (128, 64, 32,16) for the upper 4-bit.
- MOV: to transfer data between registers.
- ADDI and SUBI: for arithmetic operations, addition and subtraction.
- MOVR and MOVRHS: to control motor movement for full-step and half-step.
- BR and BRZ: to implement looping behaviour unconditionally and conditionally.

A circuit was created based on the provided diagram in the lab manual, and it will be properly connected in the software in the Lab report. The prelab report will only show the completed circuit design.

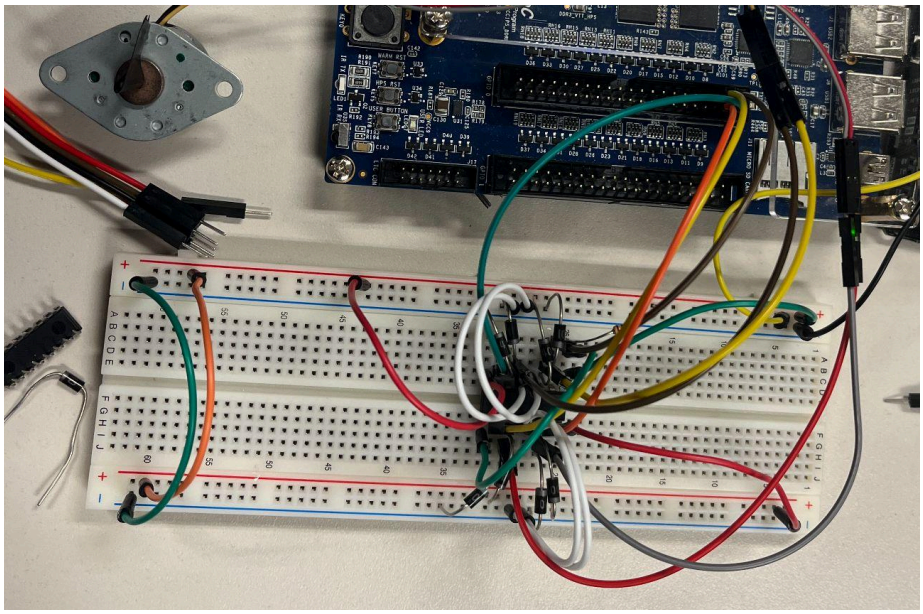


Figure 50. Image of the constructed circuit on the breadboard.

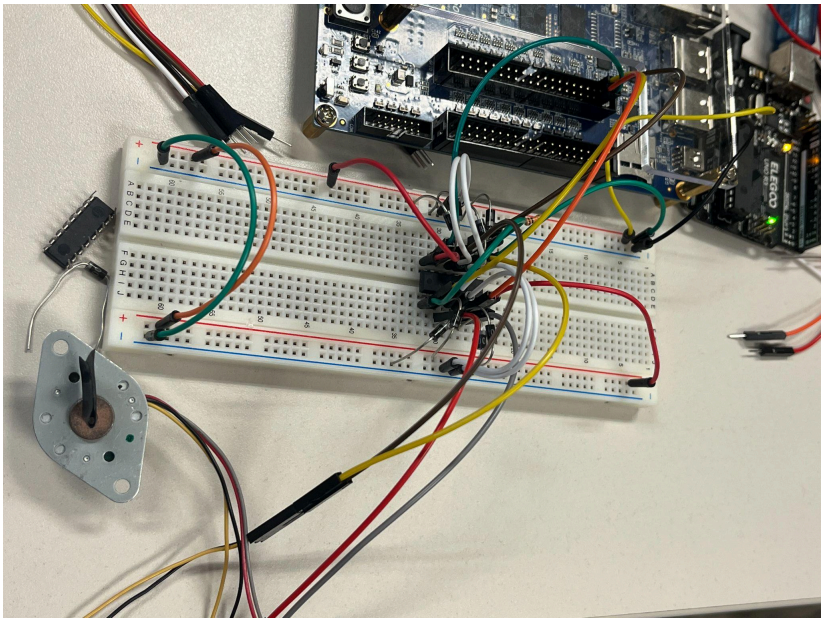


Figure 51. Image of the circuit with the motor.

Part 1: Student Number Generation

The student number generation does not move the motor and only programs the registers. It first utilizes the last two digits of the student number input into register R1 and then adds 5 to the number and moves the result into register R0. The decimal values must be converted to binary before loading, but since our student number 07 is below the lower-bit limit, only SR0 is used to construct the 8-bit value. The studentNumber.txt assembly language file and the resulting studentNumber.mif file after running the compiler.py is shown below.

```
15   ; Let's see some code (make sure add >+3 or sub <-4)
16
17   CLR r0       ; clear R0
18   CLR r1       ; clear R1
19
20   SR0 #7       ; put decimal 7 into r0
21   MOV r1 r0    ; copy 7 into r1 from r0
22
23   ADDI r1 #3   ; R1 = R1 + 3 = 10
24   ADDI r1 #2   ; R1 = R1 + 2 = 12
25
26   MOV r0 r1    ; moves r1 into r0
27
28   ; To Compile Me:
29   ; python compiler.py studentNumber.txt
```

Figure 52. Screenshot of the code for student number generation in Assembly.

```
PS C:\Users\Cynth\Documents\Tut5\lab5_compiler\lab5_compiler> python compiler.py studentNumber.
txt
            Memory Initialization File Successfully Generated!
                Ouput File Name 'studentNumber.mif'
```

Figure 53. Screenshot of the successful compilation.

```
-- Copyright (C) 2017  Intel Corporation. All rights reserved.
-- Your use of Intel Corporation's design tools, logic functions
-- and other software and tools, and its AMPP partner logic
-- functions, and any output files from any of the foregoing
-- (including device programming or simulation files), and any
-- associated documentation or information are expressly subject
-- to the terms and conditions of the Intel Program License
-- Subscription Agreement, the Intel Quartus Prime License Agreement,
-- the Intel FPGA IP License Agreement, or other applicable license
-- agreement, including, without limitation, that your use is for
-- the sole purpose of programming logic devices manufactured by
-- Intel and sold by Intel or its authorized distributors.  Please
-- refer to the applicable agreement for further details.

-- Custom Python generated Memory Initialization File (.mif)

WIDTH=8;
DEPTH=256;

ADDRESS_RADIX=UNS;
DATA_RADIX=BIN;

CONTENT BEGIN
        0       :       01100000;
        1       :       01100001;
        2       :       01000111;
        3       :       01110100;
        4       :       00001101;
        5       :       00001001;
        6       :       01110001;
        [7..255]        :       00000000;
END;
```

Figure 54. Screenshot of the successful compilation studentNumber.mif

The studentNumber.mif was inserted into the instruction_rom.mif with the datapath.v as the top-level module to generate the following waveform simulation plot.

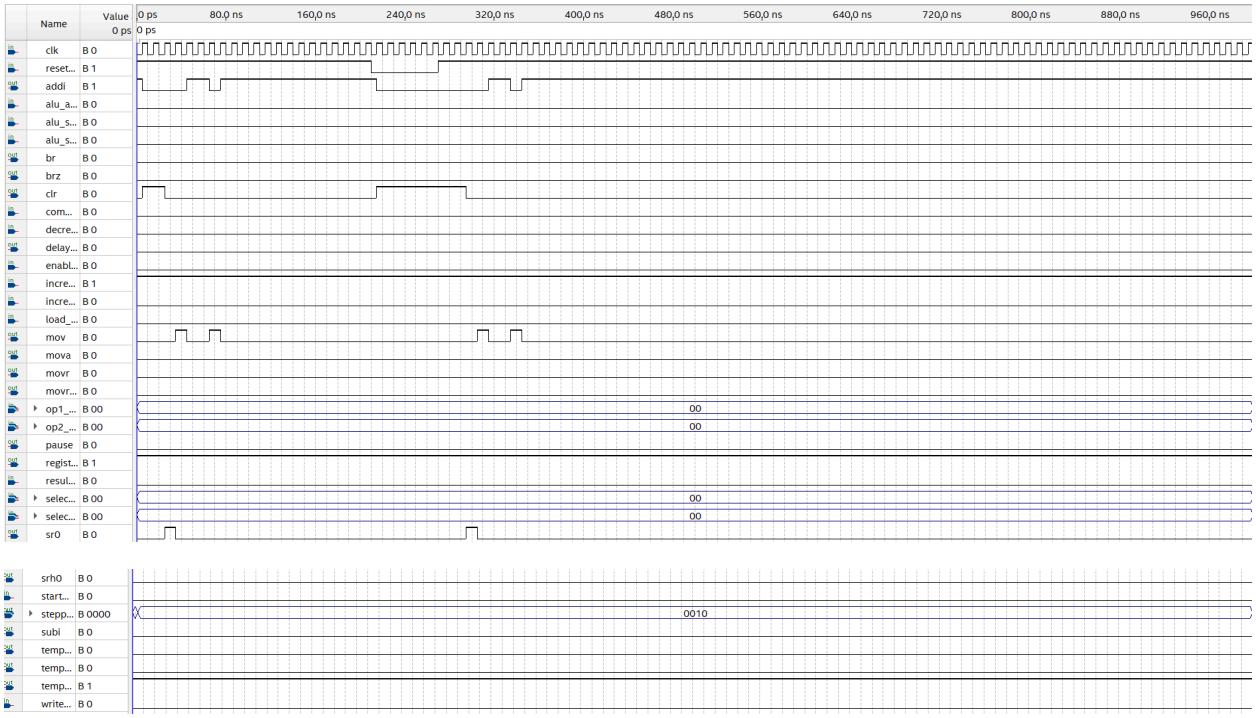


Figure 55. Waveform generated from using studentNumber.mif

Part 2: Motor Control

The motor control program Spin3.txt in assembly is used to spin the motor 3 steps clockwise, and 4 half-steps counter-clockwise with a ½ delay between each step. The program first sets the pause delay register to set the ½ delay between each step. It moves the motor for full-steps in the clockwise direction using MOVR and setting the signed binary value to positive 3. For the counter-clockwise direction in half-steps, it uses MOVHRS with a signed negative 4 value. The prompt also requires it to loop repeatedly, which is achieved by setting BR to -9, where -9 represents jumping back 9 steps at the very start of moving the motor 3 steps counter-clockwise. A generated waveform demonstrates that the program is working appropriately.

```
15   ; Let's see some code
16
17   ; pause delay set in register 0 that gets called upon whenever movr is called
18   CLR r0          ; clear Register 0 on start
19   SR0 #2          ; Set lower half of Register 0 to b0010
20   SRH0 #3         ; Sets upper half of register 0 to b0011
21                   ; Overall sets it to 50/100 used for pause delay
22   MOV r3 r0       ; copies 50 into r3 for pause delay.
23
24   ; moves motors 3 steps clockwise
25   CLR r0          ; clear Register 0 on start
26   SR0 #3          ; Set lower half of Register 0 to b0011
27   MOV r1 r0       ; Move the Register 0 to Register 1
28   MOVR r1         ; Move the motor by the value in Register 1
29
30   ; moves motor 4 steps counter-clockwise
31   CLR r0
32   SR0 #12         ; Set lower half of Register 0 to b1100
33   SRH0 #15        ; Sets upper half of register 0 to b1111
34   MOV r1 r0       ; Move the Register 0 to Register 1
35   MOVRHS r1       ; Move the motor by the value in Register 1
36
37   ; creates a loop where it jumps back to moving motor 3 steps clockwise
38   BR #-9          ; Jump backward 9 instructions
39
40   ; To Compile Me:
41   ; python compiler.py Spin3.txt
```

Figure 56. Screenshot of the code for Spin3 in Assembly.

```
PS C:\Users\Cynth\Documents\Tut5\lab5_compiler\lab5_compiler> python compiler.py Spin3.txt
            Memory Initialization File Successfully Generated!
                Ouput File Name 'Spin3.mif'
```

Figure 57. Screenshot of the successful compilation.

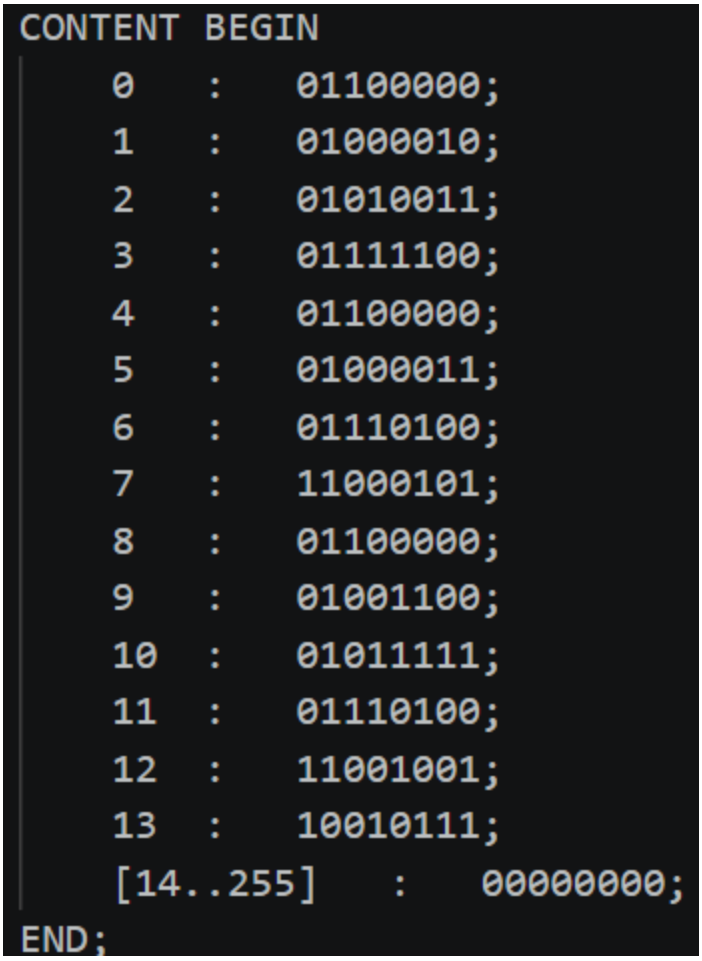


Figure 58. Assembly code for Spin3.mif.

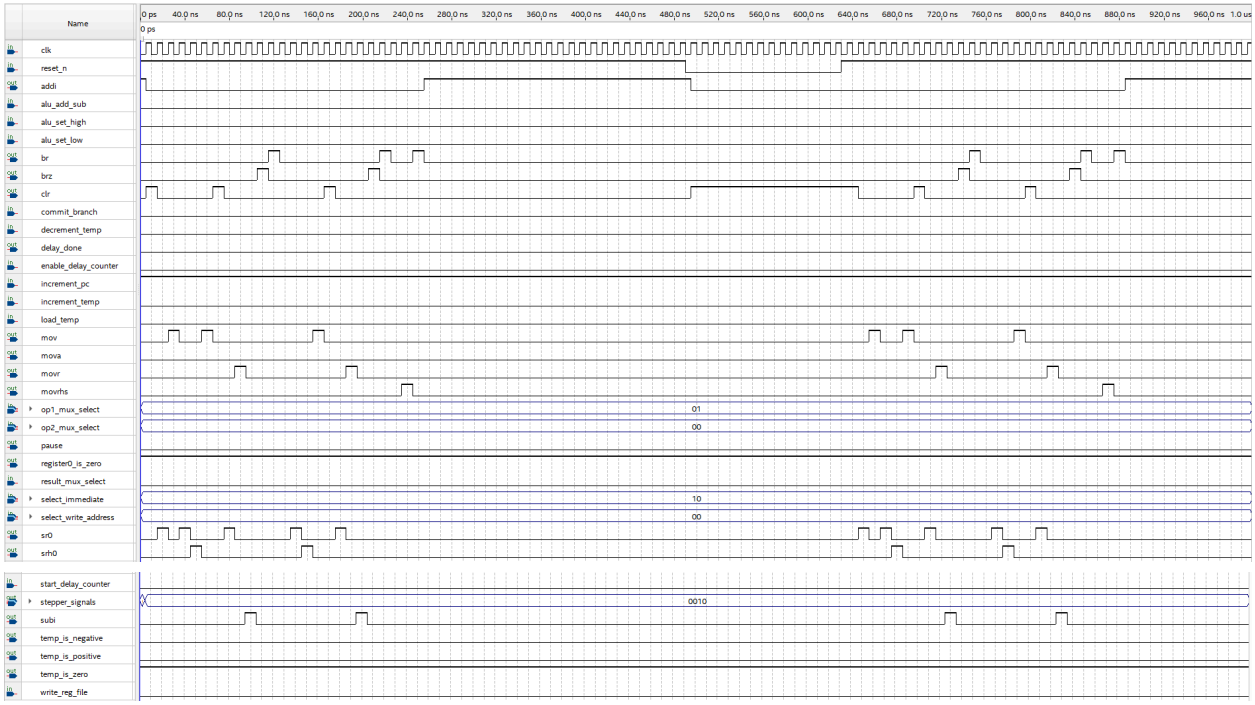


Figure 59. Waveform generated from using Spin3.mif

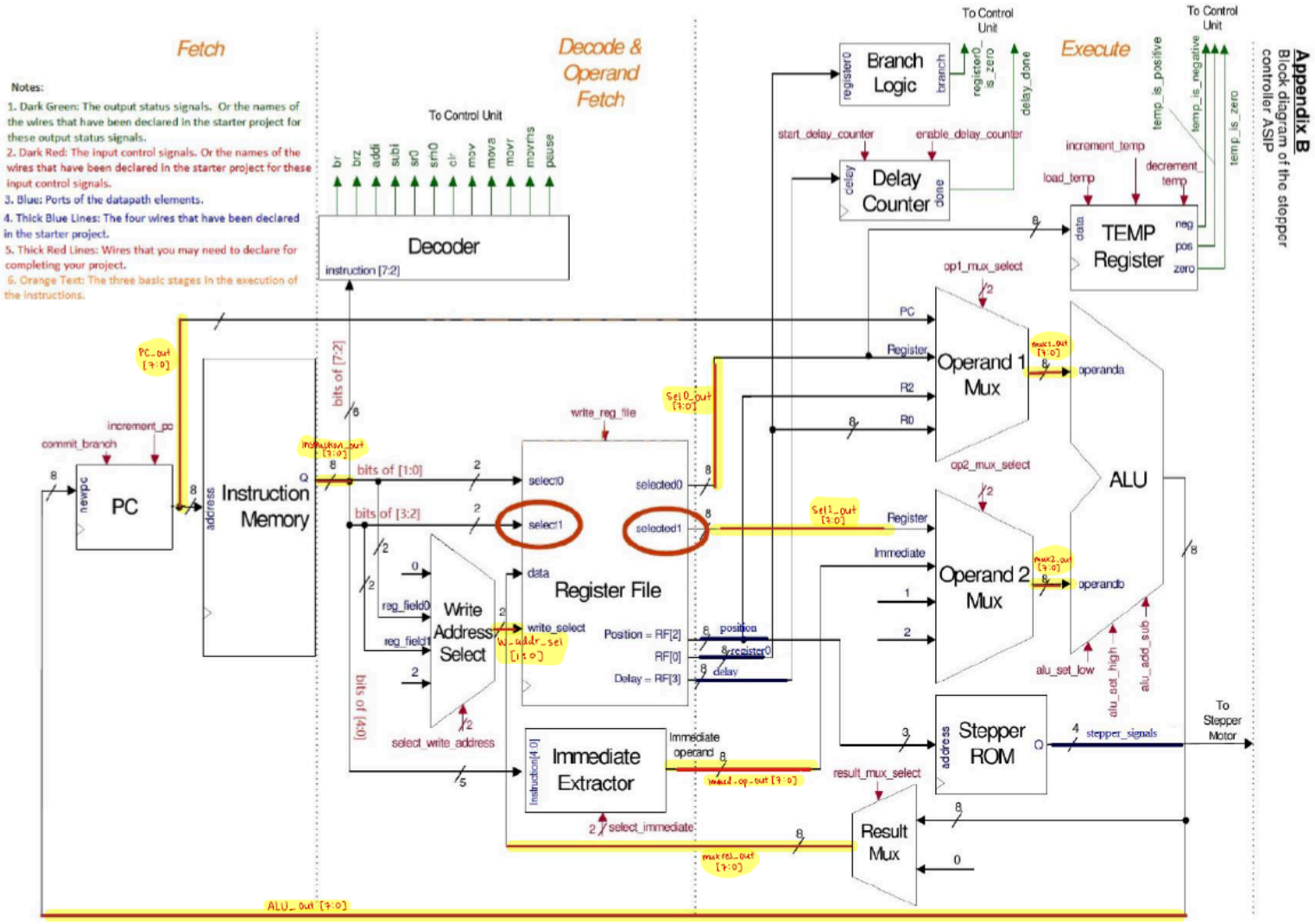


Figure 60. Labelled block diagram of the stepper controller ASIP

Code

Lab Tutorial 5

Program Counter (PC)

```
module pc (input clk, reset_n, branch, increment, input [7:0] newpc,
                        output reg [7:0] pc);

        parameter RESET_LOCATION = 8'h00;

        always@(posedge clk or negedge reset_n) begin

                if(!reset_n)
                        pc <=RESET_LOCATION;

                else if(branch)
                        pc <= newpc;

                else if(increment)
                        pc <= pc + 8'b00000001;

        end

endmodule
```

Instruction Memory

```
// synopsys translate_off
`timescale 1 ps / 1 ps
// synopsys translate_on
module instruction_rom (
	address,
	clock,
	q);

	input	[7:0]  address;
	input	  clock;
	output	[7:0]  q;
`ifndef ALTERA_RESERVED_QIS
// synopsys translate_off
`endif
	tri1	  clock;
`ifndef ALTERA_RESERVED_QIS
// synopsys translate_on
`endif

	wire [7:0] sub_wire0;
	wire [7:0] q = sub_wire0[7:0];

	altsyncram	altsyncram_component (
				.address_a (address),
				.clock0 (clock),
				.q_a (sub_wire0),
				.aclr0 (1'b0),
				.aclr1 (1'b0),
				.address_b (1'b1),
				.addressstall_a (1'b0),
				.addressstall_b (1'b0),
				.byteena_a (1'b1),
				.byteena_b (1'b1),
				.clock1 (1'b1),
				.clocken0 (1'b1),
				.clocken1 (1'b1),
				.clocken2 (1'b1),
				.clocken3 (1'b1),
				.data_a ({8{1'b1}}),
				.data_b (1'b1),
				.eccstatus (),
				.q_b (),
				.rden_a (1'b1),
```

```
                .rden_b (1'b1),
                .wren_a (1'b0),
                .wren_b (1'b0));
    defparam
        altsyncram_component.address_aclr_a = "NONE",
        altsyncram_component.clock_enable_input_a = "BYPASS",
        altsyncram_component.clock_enable_output_a = "BYPASS",
        altsyncram_component.init_file = "instruction_rom.mif",
        altsyncram_component.intended_device_family = "Cyclone V",
        altsyncram_component.lpm_hint = "ENABLE_RUNTIME_MOD=NO",
        altsyncram_component.lpm_type = "altsyncram",
        altsyncram_component.numwords_a = 256,
        altsyncram_component.operation_mode = "ROM",
        altsyncram_component.outdata_aclr_a = "NONE",
        altsyncram_component.outdata_reg_a = "UNREGISTERED",
        altsyncram_component.widthad_a = 8,
        altsyncram_component.width_a = 8,
        altsyncram_component.width_byteena_a = 1;


endmodule
```

Instruction_rom.mif for instruction_rom.v


```
-- Custom Python generated Memory Initialization File (.mif)

WIDTH=8;
DEPTH=256;

ADDRESS_RADIX=UNS;
DATA_RADIX=BIN;

CONTENT BEGIN
        0       :       01100000;
        1       :       01000010;
        2       :       01010011;
        3       :       01111100;
        4       :       01100000;
        5       :       01000011;
        6       :       01110100;
        7       :       11000001;
        8       :       11111111;
        9       :       01100000;
        10      :       01001100;
        11      :       01011111;
        12      :       01110100;
        13      :       11000001;
        14      :       11111111;
        15      :       10010101;
        [16..255]       :       00000000;
END;
```

Decoder

```
module decoder (input [7:0] instruction,
                                output br, brz, addi, subi, sr0, srh0, clr, mov, mova, movr, movrhs,
pause
);

// input can't produce waveform at 7:2 only 5:0

assign br = (instruction[7:5] == 3'b100);
assign clr = (instruction[7:2] == 6'b011000);
assign brz = (instruction[7:5] == 3'b101);
assign addi = (instruction[7:5] == 3'b000);
assign subi = (instruction[7:5] == 3'b001);
assign sr0 = (instruction[7:4] == 4'b0100);
assign srh0 = (instruction[7:4] == 4'b0101);
assign mov = (instruction[7:4] == 4'b0111);
assign mova = (instruction[7:2] == 6'b110000);
assign movr = (instruction[7:2] == 6'b110001);
assign movrhs = (instruction[7:2] == 6'b110010);
assign pause = (instruction[7:0] == 8'b11111111);


endmodule
```

Register File

```
module regfile (input clk, reset_n, write, input [7:0] data, input [1:0] select0, select1, wr_select,
                                output reg [7:0] selected0, selected1, output [7:0] delay, position,
register0);

        // The comment /* synthesis preserve */ after the declaration of a register
        // prevents Quartus from optimizing it, so that it can be observed in simulation
        // It is important that the comment appear before the semicolon
        reg [7:0] reg0 /* synthesis preserve */;
        reg [7:0] reg1 /* synthesis preserve */;
        reg [7:0] reg2 /* synthesis preserve */;
        reg [7:0] reg3 /* synthesis preserve */;

        assign register0 = reg0;
        assign position = reg2;
        assign delay = reg3;

        always @(posedge clk or negedge reset_n) begin
                if (!reset_n) begin
                        reg0 <=8'b0;
                        reg1 <=8'b0;
                        reg2 <=8'b0;
                        reg3 <=8'b0;
                end
                else if (write) begin
                        case (wr_select)
                                2'b00: reg0 <=data;
                                2'b01: reg1 <=data;
                                2'b10: reg2 <=data;
                                2'b11: reg3 <=data;
                        endcase
                end
        end

        always @(*) begin
                case (select0)
                        2'b00: selected0 <=reg0;
                        2'b01: selected0 <=reg1;
                        2'b10: selected0 <=reg2;
                        2'b11: selected0 <=reg3;
                endcase
                case (select1)
                        2'b00: selected1 <=reg0;
```

```
                2'b01: selected1 <=reg1;
                2'b10: selected1 <=reg2;
                2'b11: selected1 <=reg3;
        endcase
    end

endmodule
```

Write Address Select

```
module write_address_select (input [1:0] select, input [1:0] reg_field0, reg_field1, output reg
[1:0] write_address);

    always @(*) begin
        case(select)
            2'b00: write_address = 2'b0;
            2'b01: write_address = reg_field0;
            2'b10: write_address = reg_field1;
            2'b11: write_address = 2'b10;
        endcase
    end

endmodule
```

Result Mux

```
module result_mux (
	input select_result,
	input [7:0] alu_result,
	output reg [7:0] result
);

	always @(*) begin
		case(select_result)
			1'b0: result = alu_result;
			1'b1: result = 8'b0;
		endcase
	end



endmodule
```

Immediate Extractor

```
module immediate_extractor (input [4:0] instruction, input [1:0] select, output reg [7:0]
immediate);

	always @(*) begin
		case (select)
			2'b00: begin
				immediate[7:0] = {5'b00000, instruction[4:2]};
			end
			2'b01: begin
				immediate[7:0] = {4'b0000, instruction[3:0]};
			end
			2'b10: begin
				immediate[7:0] = {instruction[4], instruction[4],
instruction[4], instruction[4:0]};
			end
			2'b11: begin
				immediate[7:0] = 8'b0;
			end
			default: begin
				immediate[7:0] = 8'b0;
			end
		endcase
	end

endmodule
```

ALU

```
module alu (input add_sub, set_low, set_high, input [7:0] operanda , operandb, output reg [7:0]
result);

	always @(*) begin
		result = 8'b0;
		if (set_low) begin
			result [7:4] = operanda[7:4];
			result [3:0] = operandb[3:0];
		end
		else if (set_high) begin
			result[7:4] = operandb[3:0];
			result[3:0] = operanda[3:0];
		end
		else if(add_sub==1'b0) begin
			result = operanda + operandb;
		end
		else if (add_sub==1'b1) begin
			result = operanda- operandb;
		end
	end



endmodule
```

Operand 1 Mux

```
module op1_mux (input [1:0] select, input [7:0] pc, register, register0, position,
                                    output reg [7:0] result);
        always @(*) begin
                case(select)
                        2'b00: result = pc;
                        2'b01: result = register;
                        2'b10: result = position;
                        2'b11: result = register0;
                endcase
        end


endmodule
```

Operand 2 Mux

```
module op2_mux (input [1:0] select, input [7:0] register, immediate,
                                    output reg [7:0] result);

        always@(*) begin
                case (select)
                        2'b00: result = register;
                        2'b01: result = immediate;
                        2'b10: result = 8'b00000001;
                        2'b11: result = 8'b00000010;
                endcase
        end

endmodule
```

Stepper ROM

```
module stepper_rom (
	address,
	clock,
	q);

	input	[2:0]  address;
	input	 clock;
	output  [3:0]  q;
`ifndef ALTERA_RESERVED_QIS
// synopsys translate_off
`endif
	tri1	 clock;
`ifndef ALTERA_RESERVED_QIS
// synopsys translate_on
`endif

	wire [3:0] sub_wire0;
	wire [3:0] q = sub_wire0[3:0];

	altsyncram	altsyncram_component (
				.address_a (address),
				.clock0 (clock),
				.q_a (sub_wire0),
				.aclr0 (1'b0),
				.aclr1 (1'b0),
				.address_b (1'b1),
				.addressstall_a (1'b0),
				.addressstall_b (1'b0),
				.byteena_a (1'b1),
				.byteena_b (1'b1),
				.clock1 (1'b1),
				.clocken0 (1'b1),
				.clocken1 (1'b1),
				.clocken2 (1'b1),
				.clocken3 (1'b1),
				.data_a ({4{1'b1}}),
				.data_b (1'b1),
				.eccstatus (),
				.q_b (),
				.rden_a (1'b1),
				.rden_b (1'b1),
				.wren_a (1'b0),
```

```
                                .wren_b (1'b0));
        defparam
                altsyncram_component.address_aclr_a = "NONE",
                altsyncram_component.clock_enable_input_a = "BYPASS",
                altsyncram_component.clock_enable_output_a = "BYPASS",
                altsyncram_component.init_file = "stepper_rom.mif",
                altsyncram_component.intended_device_family = "Cyclone V",
                altsyncram_component.lpm_hint = "ENABLE_RUNTIME_MOD=NO",
                altsyncram_component.lpm_type = "altsyncram",
                altsyncram_component.numwords_a = 8,
                altsyncram_component.operation_mode = "ROM",
                altsyncram_component.outdata_aclr_a = "NONE",
                altsyncram_component.outdata_reg_a = "UNREGISTERED",
                altsyncram_component.widthad_a = 3,
                altsyncram_component.width_a = 4,
                altsyncram_component.width_byteena_a = 1;


endmodule
```

Stepper ROM instructions for .mif


```
-- Quartus Prime generated Memory Initialization File (.mif)

WIDTH=4;
DEPTH=8;

ADDRESS_RADIX=BIN;
DATA_RADIX=BIN;

CONTENT BEGIN
    0000  :   0010;
    0001  :   0110;
    0010  :   0100;
    0011  :   0101;
    0100  :   0001;
    0101  :   1001;
    0110  :   1000;
    0111  :   1010;
END;
```

Branch Logic

```
module branch_logic (input [7:0] register0, output reg branch);
//had to change branch to output reg instead of just output for combinational logic

        always @(*) begin
```

```
                if (register0==8'b0) begin
                        branch = 1'b1;
                end
                else begin
                        branch = 1'b0;
                end
        end

endmodule
```

Delay Counter

```
module delay_counter (input clk, reset_n, start, enable, input [7:0] delay, output reg done);
parameter BASIC_PERIOD=20'd500000;   // can change this value to make delay longer

    reg [19:0] count_cycle;
    reg [7:0] count_delay;

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            count_cycle <= 20'b0;
            count_delay <= 8'b0;
            done <= 1'b0;
        end
        else if (start) begin
            count_cycle <= 20'b0;
            count_delay <= delay;
            done <= 1'b0;
        end

        else if (enable) begin
            if (count_cycle < BASIC_PERIOD - 1) begin
                count_cycle <= count_cycle + 1;
            end
            else begin
                count_cycle <= 20'b0;
                if (count_delay > 0) begin
                    count_delay <= count_delay - 1;
                end
                if (count_delay == 1) begin
                    done <= 1;
                end
                else begin
                    done <= 0;
                end
            end
        end
        else begin
            done <= 0;
        end
    end


endmodule
```

Prelab 5

Student Number Generation

```
; Welcome to compiler tutorial #1

; Syntax:
; 1. Semicolon's are comments, anything after a Semicolon
;    will be ignored during compilation
; 2. All instructions start with an instruction name, in
;    all caps. We currently support the following instructions:
;    BR,BRZ,ADDI,SUBI,SR0,SRH0,CLR,MOV,MOVA,MOVR,MOVRHS,PAUSE
; 3. Immediate constants are preceded by a pound, #
; 4. Register adresses are preceded by the letter r
; 5. Instruction parameters can be separated by a space, comma
;    or both. The compiler is not that fussy
; 6. Only one instruction is allowed per line

; Let's see some code (make sure add >+3 or sub <-4)

CLR r0      ; clear R0
CLR r1      ; clear R1

SR0 #7      ; put decimal 7 into r0
MOV r1 r0   ; copy 7 into r1 from r0

ADDI r1 #3  ; R1 = R1 + 3 = 10
ADDI r1 #2  ; R1 = R1 + 2 = 12

MOV r0 r1   ; moves r1 into r0

; To Compile Me:
; python compiler.py studentNumber.txt
```

Student generation .mif file

-- Custom Python generated Memory Initialization File (.mif)

```
WIDTH=8;
DEPTH=256;

ADDRESS_RADIX=UNS;
DATA_RADIX=BIN;

CONTENT BEGIN
        0       :       01100000;
        1       :       01100001;
        2       :       01000111;
        3       :       01110100;
        4       :       00001101;
        5       :       00001001;
        6       :       01110001;
        [7..255]        :       00000000;
END;
```

Spin 3 clockwise, spin 4 half-step counter clockwise

```
; Welcome to compiler tutorial #1

; Syntax:
; 1. Semicolon's are comments, anything after a Semicolon
;    will be ignored during compilation
; 2. All instructions start with an instruction name, in
;    all caps. We currently support the following instructions:
;    BR,BRZ,ADDI,SUBI,SR0,SRH0,CLR,MOV,MOVA,MOVR,MOVRHS,PAUSE
; 3. Immediate constants are preceded by a pound, #
; 4. Register adresses are preceded by the letter r
; 5. Instruction parameters can be separated by a space, comma
;    or both. The compiler is not that fussy
; 6. Only one instruction is allowed per line

; Let's see some code

; pause delay set in register 0 that gets called upon whenever movr is called
CLR r0          ; clear Register 0 on start
SR0 #2          ; Set lower half of Register 0 to b0010
SRH0 #3         ; Sets upper half of register 0 to b0011
                ; Overall sets it to 50/100 used for pause delay
MOV r3 r0       ; copies 50 into r3 for pause delay.

; moves motors 3 steps clockwise
CLR r0          ; clear Register 0 on start
SR0 #3          ; Set lower half of Register 0 to b0011
MOV r1 r0       ; Move the Register 0 to Register 1
MOVR r1         ; Move the motor by the value in Register 1

; moves motor 4 steps counter-clockwise
CLR r0
SR0 #12         ; Set lower half of Register 0 to b1100
SRH0 #15        ; Sets upper half of register 0 to b1111
MOV r1 r0       ; Move the Register 0 to Register 1
MOVRHS r1       ; Move the motor by the value in Register 1

; creates a loop where it jumps back to moving motor 3 steps clockwise
BR #-9          ; Jump backward 9 instructions

; To Compile Me:
; python compiler.py Spin3.txt
```

Spin 3 clockwise, spin 4 half-step counter clockwise .mif file

-- Custom Python generated Memory Initialization File (.mif)

```
WIDTH=8;
DEPTH=256;

ADDRESS_RADIX=UNS;
DATA_RADIX=BIN;

CONTENT BEGIN
        0       :       01100000;
        1       :       01000010;
        2       :       01010011;
        3       :       01111100;
        4       :       01100000;
        5       :       01000011;
        6       :       01110100;
        7       :       11000101;
        8       :       01100000;
        9       :       01001100;
        10      :       01011111;
        11      :       01110100;
        12      :       11001001;
        13      :       10010011;
        [14..255]       :       00000000;
END;
```